PROGRAMMAZIONE FUNZIONALE

Appunti del corso

Lorenzo Carlin

Anno Accademico 2025–2026

Introduzione

Questa raccolta di appunti è stata realizzata durante lo studio del corso di Programmazione Funzionale. L'obiettivo della dispensa è raccogliere in un unico documento gli aspetti teorici e pratici affrontati durante il corso, fornendo una sintesi organica utile per lo studio, il ripasso e la preparazione dell'esame.

Argomenti trattati

- Espressioni e tipi
- Type system
- Errori, variabili e funzioni
- Regole di scope
- Astrazione dei controlli e ricorsione
- Pattern matching ed environment locali in ML
- Input/output, eccezioni e funzioni polimorfe
- Funzioni di ordine superiore e funzioni curried
- Data types
- Lambda calcolo

Nota

Gli appunti non hanno la pretesa di sostituire il materiale ufficiale del corso, ma rappresentano una rielaborazione personale dei contenuti trattati a lezione, integrata con esempi, osservazioni ed esercizi utili alla comprensione degli argomenti.

Versione compilata il 18 giugno 2026

01. Espressioni e tipi

Introduzione a ML

- ML è un linguaggio di **programmazione funzionale**: le protagoniste sono le funzioni, con le loro definizioni e applicazioni.
- In ML viene fatto massiccio uso della **ricorsione**, al posto di costrutti come i cicli `for` e `while`.
- Le regole di `if-then-else` vengono implementati tramite il **pattern matching**, ossia un insieme di regole del tipo:
  se il valore ha questa forma → fai {questo},
  altrimenti → fai {quest'altro}
- IN ML la computazione avviene tramite valutazione di espressioni e non tramite assegnazioni che modificano le variabili, ciò significa che **non sono previsti side effects**. Nonostante ciò il linguaggio mette a disposizione comandi come la stampa ed altri, ma non fanno propriamente parte del paradigma di programmazione funzionale.
- I tipi di valori e variabili sono valutati a tempo di compilazione. ML è un linguaggio **strongly-typed**, ossia prevede una notevole rigidità sui tipi.
- Una funzione può avere più firme, differenti l'una dall'altra. Questo comportamento prende il nome di **polimorfismo**.
- I **Data Types** in ML permettono di costruire nuovi tipi a partire dai tipi primitivi.

Espressioni e comandi

Le **espressioni** sono entità sintattiche la cui valutazione produce un valore oppure non termina (*undefined expression*).

Esistono due modi per valutare le espressioni:

1. **eager evaluation**, che valuta prima tutti gli operandi e solo dopo applica gli operatori
2. **lazy evaluation**, che valuta gli operandi solo quando strettamente necessario

> **Esempio**
>
> ```
> if(a && b && c) {}
> ```

> In questo caso un linguaggio come C++ applica la lazy evaluation: se `a` è `false` non va a valutare il resto dell'espressione, perché sa già che sarà per forza falsa.

Anche ML usa la lazy evaluation per i costrutti **`andalso`**, **`orelse`**, **`if-then-else`**, altrimenti usa sempre la eager evaluation.

```
> 1+2*3;
(* val it = 7: int *)
```

- `val`: valore dell'espressione
- `it`: nome del risultato
- `int`: tipo del risultato (assegnato automaticamente)

I comandi sono entità sintattiche la cui valutazione non ritorna necessariamente un valore ma può avere un side effect. Per questo motivo nella programmazione funzionale non sono previsti comandi.

Un side effect è un qualsiasi cambio nello stato del sistema. Il più comune, che si verifica nei linguaggi imperativi, è la modifica del valore di una variabile.
ML non permette di effettuare side effects!

Tipi di dati in ML

Un data type è un insieme di valori e di operatori, definiti su tali valori.

Tipi primitivi:

- `int` → interi (positivi e negativi) e esadecimali

```
> 0x124;
(* val it = 292: int *)
> ~0x124;
(* val it = ~292: int *)
```

> ⚠ **Attenzione**
>
> In ML non si utilizza il simbolo `-` per indicare il segno meno, si usa invece il simbolo `~`. `-` è usato solo come operatore di sottrazione.

- `real` → reali (scritti per intero oppure usando `E` / `e`, inteso come potenza di 10)

```
> ~123.0;
(* val it = ~123.0: real *)
```

```
> 3E~3;
(* val it = 0.003: real *)
> 3.14e12;
(* val it = 3.14E12: real *)
```

- `bool` → true o false

```
> true;
(* val it = true: bool *)
> 1=3;
(* val it = false: bool *)
```

- `char` → singoli caratteri, indicati come `#"{carattere}"`

```
> #"a";
(* val it = #"a": char *)
```

- `string` → stringhe, racchiuse tra doppi apici `" "`

```
>"foo";
(* val it = "foo": string *)
```

> **Osservazione**
>
> ML prevede anche dei caratteri speciali:
>
> - `\n` : newline
> - `\t` : tab
> - `\\` : backslash
> - `\"` : double-quote
> - `\xyz` : carattere ASCII con il codice `xyz`

- `unit` → assume un solo valore, ossia `()`. È usato per le espressioni che non ritornano un valore.

Operatori in ML

Operatori aritmetici

- `+` → somma
- `-` → differenza
- `*` → moltiplicazione
- `/` → divisione tra reali

- `div` → divisione tra interi, arrotonda per difetto
- `mod` → resto della divisione tra interi

> **Osservazione**
>
> Le regole di precedenza delle operazioni sono quelle comuni.

```
> 3.0 - 4.5 + 6.7;
(* val it = 5.2: real *)
> 3 - 4 * 2;
(* val it = ~5: int *)
> 43 div (8 mod 3) * 5;
(* val it = 105: int *)
> 3 + 4.0;
(* poly: : error: Type error in function application.
Function: + : int * int -> int *)
```

Operatori delle stringhe

- `^` → concatenazione tra due stringhe

```
> "house" ^ "cat";
(* val it = "housecat": string *)
> "house" ^ "";
(* val it = "house": string *)
```

Operatori di confronto

- `=` → uguale
- `<` → minore
- `>` → maggiore
- `<=` → minore o uguale
- `>=` → maggiore o uguale
- `<>` → diverso

```
> 1 < 2;
(* val it = true: bool *)
> 1.0 < 2.0;
(* val it = true: bool *)
> 1 = 2;
(* val it = false: bool *)
> 1.0 = 2.0;
(* poly: : error: Type error in function application.
Function: = : ''a * ''a -> bool
```

```
Argument: (1.0, 2.0) : real * real
Reason: Can't unify ''a to real (Requires equality type)
Found near 1.0 = 2.0 *)
```

> ⚠ **Warning**
>
> Non sono consentite le operazioni `=` e `<>` sui numeri reali!

```
> #"Z" < #"a";
(* val it = true: bool *)
> "abc" <= "ab";
(* val it = false: bool *)
> "abc" <= "acc";
(* val it = true: bool *)
```

Operatori logici

- `not`
- `andalso`
- `orelse`

```
> 1<2 andalso 3>4;
(* val it = false: bool *)
> 1<2 orelse 3>4;
(* val it = true: bool *)
```

> 👁 **Osservazione**
>
> `not` ha la precedenza sugli altri due, `andalso` ha la precedenza su `orelse`.

If-then-else

Sintassi: `if <p> then <exp1> else <exp2>`. L'`else` è obbligatorio.

```
> if 1<2 then 3+4 else 5+6;
(* val it = 7: int *)
> if 5<6 then 5 else 6;
(* val it = 5: int *)
> if 5<6 then 5;
(* poly: : error: else expected but ; was found
poly: : error: Expression expected but ; was found
Static Errors *)
> if 5<6 then 5 else 6.0;
(* poly: : error: Type mismatch between then-part and else-part.
```

```
Then: 5 : int
Else: 6.0 : real
Reason: Can't unify int to real (Incompatible overloadings)
Found near if 5 < 6 then 5 else 6.0
Static Errors *)
```

02. Type system, errori, variabili e funzioni

Type systems

Il type system di un linguaggio comprende:

1. I tipi predefiniti
2. I meccanismi per definire nuovi tipi
3. La specificazione se i tipi sono controllati staticamente o dinamicamente
4. I meccanismi di controllo (equivalenza, compatibilità, inferenza)

Tipi semplici vs tipi composti

- Tipi semplici (o scalari): sono i tipi primitivi, non derivano da altri tipi
- Tipo composti: ottenuti dalla combinazione di altri tipi, usando appositi costruttori
  Per definire un nuovo tipo si usa la sintassi `type newtype = expression;`

Controllo dei tipi

- Statico: avviene a tempo di compilazione, prima di arrivare all'utente. Dunque è più efficiente a runtime, ma il design è più complesso e la compilazione richiede più tempo.
- Dinamico: avviene durante l'esecuzione del codice. Identifica gli errori di tipo, ma lo fa soltanto a runtime e non è efficiente.

Linguaggi Strongly typed vs linguaggi Weakly typed

- Strongly typed: il linguaggio è molto rigido sui tipi, ma ciò garantisce che il tipo di una variabile non cambi in modo inaspettato (es. conversioni implicite).
- Weakly typed: il linguaggio è più rilassato sulla manipolazione di tipi, ma il tipo di una variabile può cambiare in modo inaspettato.

> ⚠ **Attenzione**
>
> ML è un linguaggio strongly typed che controlla i tipi in modo statico.

Type safety

Un type system è type safe se non è possibile che alcuni errori di tipo non vengano scovati. I linguaggi strongly typed hanno un alto grado di type safety.
C e C++, per fare due nomi conosciuti, sono type unsafe!

Meccanismi di controllo

Equivalenza di tipo

Due tipi sono equivalenti se ogni oggetto del primo tipo appartiene anche al secondo tipo e viceversa.
Esistono due regole per l'equivalenza di tipo:

1. **Equivalenza per nome**: definizione del tipo opaca
   Due tipo sono equivalenti per nome se hanno lo stesso nome
2. **Equivalenza strutturale**: definizione del tipo trasparente
   Due tipi sono equivalenti se hanno la stessa struttura

Compatibilità e conversione

`T` è compatibile con `S` se gli oggetti di tipo `T` possono essere usati in contesti dove ci si aspetta oggetti di tipo `S`.

> **Esempio**
>
> ```
> int n;
> float r;
> r = r + n;
> ```
>
> In alcun linguaggi questo si può fare, in quei casi `int` e `float` sono compatibili.

La compatibilità è riflessiva e transitiva ma non è simmetrica.

Se `T` è compatibile con `S`, allora esiste qualche meccanismo di **conversione** tra i due.
I 2 principali metodi sono:

- **Conversione implicita** (coercizione), non menzionata nel programma
- **Conversione esplicita** (cast), menzionata nel programma
  I linguaggi moderni preferiscono il cast alla coercizione.

Type errors in ML

```
> 1 + 2
(* val it = 3: int *)
> 1.0 + 2.0;
(* val it = 3.0: real *)
> 1 + 2.0;
(* poly: : error: Type error in function application.
Function: + : int * int -> int
Argument: (1, 2.0) : int * real
Reason:
```

```
Can't unify int (*In Basis*) with real (*In Basis*)
(Different type constructors)
Found near 1 + 2.0
Static Errors *)
> 1.0 + 2;
(* poly: : error: Type error in function application.
Function: + : real * real -> real
Argument: (1.0, 2) : real * int
Reason:
Can't unify int (*In Basis*) with real (*In Basis*)
(Different type constructors)
Found near 1.0 + 2
Static Errors *)
```

> ⚠ **Attenzione**
>
> L'operatore `+` può avere più tipi, ma ML si basa sul primo argomento passato per dichiarare l'errore, che si pone perché il secondo argomento non corrisponde al primo.

```
> #"a" ^ "bc";
(* poly: : error: Type error in function application.
Function: ^ : string * string -> string
Argument: (#"a", "bc") : char * string *)
> 1/2;
(* poly: : error: Type error in function application.
Function: / : real * real -> real
Argument: (1, 2) : int * int *)
> if 1<2 then #"a" else "bc";
(* poly: : error: Type mismatch between then-part and else-part.
Then: #"a" : char
Else: "bc" : string *)
```

Conversione di tipi in ML

Non esiste la conversione automatica dei tipi in ML, deve essere fatta esplicitamente. A questo proposito sono fornite una serie di funzioni apposite.

Conversione tra interi e reali

- `real: int -> real`
- `floor: real -> int`, arrotonda per difetto
- `ceil: real -> int`, arrotonda per eccesso
- `round: real -> int`, arrotonda all'intero più vicino
- `trunc: real -> int`, tronca

```
> real(4);
(* val it = 4.0: real *)
> real 4;
(* val it = 4.0: real *)
> floor(3.5);
(* val it = 3: int *)
> ceil(3.5);
(* val it = 4: int *)
> round(3.5);
(* val it = 4: int *)
> trunc(3.5);
(* val it = 3: int *)
> floor(~3.5);
(* val it = ~4: int *)
> trunc(~3.5);
(* val it = ~3: int *)
```

Conversione tra caratteri e interi

- `ord: char -> int`
- `chr: int -> char`

```
> ord #"a";
(* val it = 97: int *)
> ord #"a" - ord #"A";
(* val it = 32: int *)
> chr 97;
(* val it = #"a": char *)
```

Conversione da caratteri a stringhe

- `str: char -> string`

```
> str #"a";
(* val it = "a": string *)
```

Nomi, Oggetti denotabili e variabili

Un nome è una sequenza di caratteri usata per denotare qualcos'altro.
Un oggetto denotabile è un oggetto a cui può essere associabile un nome.
Il binding è l'associazione tra un nome e un oggetto denotabile. Tali associazioni non sono fissate una volta per tutte.
L'environment è la collezione di tutte le associazioni tra nomi e oggetti denotabili a runtime a uno specifico punto del programma in uno specifico momento dell'esecuzione.
La dichiarazione è il meccanismo (implicito o esplicito) che crea le associazioni.

> **Osservazione**
>
> Lo stesso nome può indicare oggetti denotabili differenti in punti diversi del programma. Uno stesso oggetto può avere più nomi (alias) associati in punti diversi del codice.

Una variabile è un oggetto che può assumere dei valori all'interno di un dominio.

> **Warning**
>
> In ML, il valore di una variabile non può essere modificato.

Variabili

Nei linguaggi di programmazione funzionali, come ML, una variabile è un identificatore che rappresenta un valore non modificabile, come in matematica.
L'environment di ML è l'insieme delle coppie `(identificatore, valore)` e viene modificato tramite le val-declarations (~ assegnamenti):

- `val <name> = <value>;`
- `val <name>:<type> = <value>;`
- `val <name> = <expression>;`
  Questo comando crea un'associazione tra un nome e una variabile.

```
> val pi = 3.14159;
(* val pi = 3.14159: real *)
> val v = 10.0/2.0;
(* val v = 5.0: real *)
```

> **Osservazione**
>
> Non è necessario specificare il tipo delle variabili e queste ultime non possono essere modificate.

Se avviene una nuova dichiarazione di una variabile con lo stesso nome di una già precedentemente dichiarata, esisteranno due versioni diverse di tale variabile e, quando si proverà ad accedere alla variabile in questione, si accederà alla seconda. Ma la prima rimane in memoria.

```
> val pi = 3.14159;
(* val pi = 3.14159: real *)
> val radius = 4.0;
```

```
(* val radius = 4.0: real *)
> pi * radius * radius;
(* val it = 50.26544: real *)
> val area = pi * radius * radius;
(* val area = 50.26544: real *)
```

Tipi complessi in ML

Tuple

```
> val t = (4, 5.0, "six");
(* val t = (4, 5.0, "six"): int * real * string *)
```

> **Osservazione**
>
> Una tupla può contenere al suo interno valori dai tipi diversi tra di loro.

```
> val t = (4, 5.0, "six");
(* val t = (4, 5.0, "six"): int * real * string *)
> val t = (1, (2, 3, 4));
(* val t = (1, (2, 3, 4)): int * (int * int * int) *)
```

Per accedere alle componenti di una tupla si usa `#`

```
> val t = (4, 5.0, "six");
(* val t = (4, 5.0, "six"): int * real * string *)
> #1 (t);
(* val it = 4: int *)
> #3 (t);
(* val it = "six": string *)
> #4 (t);
(* poly: : error: Type error in function application.
Function: #4 : 'a * 'b * 'c * 'd -> 'd
Argument: (t) : int * real * string
Reason: Can't unify {4: 'a} to int * real * string (Field 4 missing)
Found near #4 (t)
Static Errors *)
```

Liste

Si rappresentano con le parentesi quadre. Tutti gli elementi di una lista devono essere dello stesso tipo.

```
> [1,2,3];
(* val it = [1, 2, 3]: int list *)
```

```
> [1.0,2.0];
(* val it = [1.0, 2.0]: real list *)
> [1,2.0];
(* poly: : error: Elements in a list have different types.
Item 1: 1 : int
Item 2: 2.0 : real
Reason:
Can't unify int (*In Basis*) with real (*In Basis*)
(Different type constructors) *)
> [];
(* val it = []: 'a list *)
```

> **Osservazione**
>
> Il tipo `'a` è un tipo generico: viene specificato solamente quando viene utilizzato.

- `hd` → primo elemento della lista

```
> val L = [2,3,4];
(* val L = [2, 3, 4]: int list *)
> val M = [5];
(* val M = [5]: int list *)
> hd(L);
(* val it = 2: int *)
> hd(M);
(* val it = 5: int *)
```

- `tl` → tutto ciò che non è il primo elemento della lista

```
> L;
(* val it = [2, 3, 4]: int list *)
> M;
(* val it = [5]: int list *)
> tl (L);
(* val it = [3, 4]: int list *)
> tl (M);
(* val it = []: int list *)
```

Si possono concatenare le liste con `@` (le due liste devono essere dello stesso tipo).

```
> [1,2] @ [3,4];
(* val it = [1, 2, 3, 4]: int list *)
> [1,2] @ ["a","b"];
(* poly: : error: Type error in function application.
Function: @ : int list * int list -> int list
```

```
Argument: ([1, 2], ["a", "b"]) : int list * string list
Reason:
Can't unify int (*In Basis*) with string (*In Basis*)
(Different type constructors) *)
```

Con `::` si può combinare un elemento e una lista dello stesso tipo. Questa operazione è associativa.

```
> 2 :: [3,4];
(* val it = [2, 3, 4]: int list *)
> 2 :: nil;
(* val it = [2]: int list *)
> 2 :: [];
(* val it = [2]: int list *)
> 1 :: 2 :: 3 :: nil;
(* val it = [1, 2, 3]: int list *)
> (1 :: (2:: (3::nil)));
(* val it = [1, 2, 3]: int list *)
```

> **Osservazione**
>
> `nil` corrisponde ad una lista vuota.

Con `explode` si trasforma una stringa in una lista di caratteri.

```
> explode ("abcd");
(* val it = [#"a", #"b", #"c", #"d"]: char list *)
> explode ("");
(* val it = []: char list *)
```

E con `implode` si trasforma una lista di caratteri in una stringa.

```
> implode ([ #"a", #"b", #"c", #"d"]);
(* val it = "abcd": string *)
> implode (nil);
(* val it = "": string *)
> implode (explode ("xyz"));
(* val it = "xyz": string *)
```

Il type system di ML

- Tipi primitivi: `int`, `real`, `bool`, `char`, `string`, `unit`
- Tipi complessi: 2 costruttori
  - `T1 * T2 * ... * Tn (tuples)`

- `T list`

```
> [1, 2, 3];
(* val it = [1, 2, 3]: int list *)
> ("ab", [1,2,3], 4);
(* val it = ("ab", [1, 2, 3], 4): string * int list * int *)
> [[(1,2),(3,4)], [(5,6)], nil];
(* val it = [[(1, 2), (3, 4)], [(5, 6)], []]: (int * int) list list *)
```

Funzioni in ML

Le funzioni (rappresentate come espressioni parametrizzare) sono un altro tipo di valore in ML.

La sintassi è `fn <param> => <expression>;`, ma si può anche applicare direttamente la funzione al parametro richiesto:

```
(fn n => n+1)5;
(* val it = 6: int *)
```

Si può associare le funzioni ad un nome, come se fossero valori:

```
> val increment = fn n => n+1;
(* val increment = fn: int -> int *)
```

Esiste anche una notazione sintattica semplificata per le funzioni con nome:

```
> fun increment n = n+1;
(* val increment = fn: int -> int *)
> increment 5;
(* val it = 6: int *)
```

In caso di ambiguità, ML assegna dei tipi di default. Ad esempio, nella prima funzione `square(x)` il tipo assegnato è `int`, che vince su `real` solo per il fatto che è il tipo di default.
È possibile specificare esplicitamente i tipi di una funzione:

```
> fun square (x) = x * x;
(* val square = fn: int -> int *)
> fun square (x:real) = x * x;
(* val square = fn: real -> real *)
```

Tutte le funzioni in ML hanno un solo parametro, ma questo può essere una tupla.

Esempio

Scriviamo una funzione che trova il massimo di tre numeri reali.

```
> fun max3(a:real,b,c) =
    if a>b then
        if a>c then a else c
        else
            if b>c then b else c;
(* val max3 = fn: real * real * real -> real *)
> max3(5.0,4.0,7.0);
(* val it = 7.0: real *)
```

03. Laboratorio 1

Esercizio 1.1

Scrivere l'espressione per convertire:

1. `123.45` nel numero intero inferiore più vicino
2. `-123.45` nel numero intero inferiore più vicino
3. `123.45` nel numero intero superiore più vicino
4. `-123.45` nel numero intero superiore più vicino

```
> floor(123.45);
(* val it = 123: int *)

> floor(~123.45);
(* val it = ~124: int *)

> ceil(123.45);
(* val it = 124: int *)

> ceil(~123.45);
(* val it = ~123: int *)
```

Esercizio 1.2

Scrivere l'espressione per convertire:

1. `#"Y"` a un intero
2. `120` a un carattere
3. `97.0` a un carattere
4. `#"N"` a un reale
5. `#"Z"` a una stringa

```
> ord(#"Y");
(* val it = 89: int *)

> chr(120);
(* val it = #"x": char *)
```

```
> chr(floor(97.0));
(* val it = #"a": char *)

> real(ord(#"N"));
(* val it = 78.0: real *)

> str(#"Z");
(* val it = "Z": string *)
```

Esercizio 1.3

Come possiamo fixare gli errori in queste espressioni?

1. `ceil(4);`
2. `if true then 5+6 else 7.0;`

```
> ceil(4.0);
(* val it = 4: int *)

> if true then 5+6 else 7;
(* val it = 11: int *)
```

Esercizio 1.4

Come possiamo fixare gli errori in queste espressioni?

1. `if 0 then 1 else 2;`
2. `ord("a")`

```
> if false then 1 else 2;
(* val it = 2: int *)

> ord(#"a");
(* val it = 97: int *)
```

Esercizio 1.5

Fixa le seguenti espressioni:

1. `explode ["bar"];`
2. `implode (#"a", #"b");`
3. `["r"]::["a", "t"];`

```
> explode "bar";
(* val it = [#"b", #"a", #"r"]: char list *)

> implode [#"a", #"b"];
(* val it = "ab": string *)

> "r"::["a", "t"];
(* val it = ["r", "a", "t"]: string list *)
```

Esercizio 1.6

Dai esempi di questi tipi di oggetti, senza usare liste vuote:

1. `int list list list`
2. `(int * char) list`
3. `string list * (int * (real * string)) * int`

```
> [[[1, 2], [3, 4]], [[5, 6], [7, 8]]];
(* val it = [[[1, 2], [3, 4]], [[5, 6], [7, 8]]]: int list list list *)

> [(1, #"a"), (2, #"b"), (3, #"c")];
(* val it = [(1, #"a"), (2, #"b"), (3, #"c")]: (int * char) list *)

> (["ciao", "amico"], (27, (13.10, "auguri")), 18);
(* val it = (["ciao", "amico"], (27, (13.1, "auguri")), 18): string list *
(int * (real * string)) * int *)
```

Esercizio 1.7

Dai esempi di questi tipi di oggetti, senza usare liste vuote:

1. `((int * int) * (bool list) * real) * (real * string)`
2. `(bool * int) * char`
3. `real * int list list list list`

```
> (((1, 2), [true, true, false], 3.4), (5.6, "ciao"));
(* val it = (((1, 2), [true, true, false], 3.4), (5.6, "ciao")): ((int *
int) * bool list * real) * (real * string) *)

> ((true, 1), #"a");
(* val it = ((true, 1), #"a"): (bool * int) * char *)

> (1.2, [[[1, 2], [3, 4]], [[5, 6], [6, 7]]], [[[8, 9], [10, 11]], [[12,
13], [14, 15]]]);
```

```
(* val it = (1.2, [[[[1, 2], [3, 4]], [[5, 6], [6, 7]]], [[[8, 9], [10,
11]], [[12, 13], [14, 15]]]]): real * int list list list list *)
```

04. Regole di scope

Inferenza di tipi in ML

ML si aspetta che:

- nella valutazione di un'espressione aritmetica, tutti gli operandi siano dello stesso tipo del primo valutato
- nella valutazione di un espressione di confronto, entrambi gli argomenti siano nello stesso tipo
- nella valutazione di un espressione condizionale, i tipi di `then` e di `else` devono essere uguali
- se un'espressione viene usata come argomento di una funzione, allora il parametro deve essere di quel tipo
- se l'espressione che definisce il risultato di una funzione è di un determinato tipo, allora la funzione ritorna tale tipo

> **Osservazione**
>
> Se non c'è modo di determinare il tipo degli argomenti di una funzione overloaded (ad esempio `+` e `*`, ML usa il tipo di default (per somme e prodotti è `int`).
>
> Se invece non c'è modo di determinare i tipi degli argomenti e gli operatori non vengono usati, viene usato il tipo generico `'a`. Se ci sono due argomenti di cui non possiamo determinare il tipo, che non vengono usati e che non hanno relazione tra loro, vengono usati i tipi generici `'a` e `'b`.

Environment

L'environment è l'insieme delle associazioni tra nomi e oggetti denotabili che esistono a runtime in uno specifico punto del programma in uno specifico momento dell'esecuzione.

La dichiarazione è il meccanismo (implicito o esplicito) che crea le associazioni nell'environment.

Un blocco è una sezione del programma che è identificata da un segno di apertura e uno di chiusura (`{}` in C e Java, `let ... in ... end` in ML) che contiene la dichiarazione locale a quella regione.
Esistono:

- **blocchi associati ad una procedura**: sono il corpo di una procedura
- **blocchi in-line (anonimi)**: possono apparire in una posizione qualsiasi dove ci sono dei comandi
  I blocchi possono essere annidati tra loro in ML (non in C).

L'environment in un blocco può essere suddiviso in:

- **environment locale**: variabili locali, parametri formali
- **environment non locale**: associazioni ereditate da altri blocchi esterni
- **environment globale**: contiene associazioni comuni a tutti i blocchi (ex. variabili globali)

Un environment è definito da:

1. Regole di visibilità
2. Regole di scope
3. Regole per il passaggio dei parametri
4. Politica di binding (= associazione tra nome e oggetto denotabile)

Regole di visibilità

Una dichiarazione locale in un blocco è visibile in questo blocco e in tutti quelli annidati, fino a quando non viene dichiarata un'altra variabile con lo stesso nome. In tale caso la prima dichiarazione rimane ma è nascosta/irraggiungibile.
Le dichiarazione dentro ad un blocco non sono visibili all'esterno di questo.

Regole di scope

Le regole di scope specificano meglio le regole di visibilità.
Un riferimento non locale in un blocco può essere risolto in due modi:

1. **Scope statico**: nel blocco che contiene sintatticamente il blocco corrente
2. **Scope dinamico**: nel blocco che è stato eseguito immediatamente prima del blocco corrente

Scope statico

Un nome non locale è risolto nel blocco che lo include testualmente.

```
{
      int x = 0;
      void fie(int n) {
           x = n+1;
      }
      fie(3);
      write(x);
      {
```

```
            int x = 0;
            fie(3);
            write(x);
        }
        write(x);
}
```

In questo caso, secondo lo scope statico, viene stampato `4 0 4`, perché la variabile `x` nella funzione `fie()` è sempre quella definita la riga prima della funzione, ossia nel blocco che testualmente contiene la funzione.
Se cambiassimo il nome della seconda `x` in `y`, di fatto non cambierebbe nulla.

Vantaggi:

- il programma è di più facile comprensione/lettura
- il compilatore può collegare le occorrenze del nome con la loro dichiarazione corretta (⇒ ottimizzazione del codice)
- il compilatore ha informazioni importanti sul metodo di storage delle variabili, rendendo l'esecuzione più efficiente
  ALGOL, Pascal, C/C++, Ada, Scheme, ML e Java usano scope statico.

Scope dinamico

Un nome non locale è risolto nel blocco che è stato attivato più recentemente e che non è stato ancora disattivato.

```
{
     int x = 0;
     void fie(int n) {
          x = n+1;
     }
     fie(3);
     write(x);
     {
          int x = 0;
          fie(3);
          write(x);
     }
     write(x);
}
```

Nello stesso esempio di prima adesso viene stampato `4 4 4`, perché la seconda chiamata a `fie(3)` viene applicata sulla `x` all'interno del secondo blocco.
In questo caso, se cambiassimo il nome della seconda `x` in `y`, l'output sarebbe decisamente diverso.

Vantaggi:

- si può cambiare il comportamento del programma tramite ridefinizione di variabili non locali

Svantaggi;

- rende più difficile la lettura del programma
- è meno efficiente

> **Esercizio 4.1**
>
> Considera il seguente frammento di codice scritto in pseudo-codice e che usa scope statico. `read(Y)` consente la lettura di `Y` da standard input.
> Qual è l'output del programma?

```
int X = 0;
int Y;
void fie() {
     X++;
}
void foo() {
     X++;
     fie();
}
read(Y);
if(Y > 0) {
     int X = 5;
     foo();
} else {
     foo();
}
write(X);
```

L'output è `2`, in quanto la `X` da considerare è la prima dichiarata e viene incrementata due volte: una da `fie()` e una da `foo()`.

> **Esercizio 4.2**
>
> Considera il seguente frammento di codice scritto in pseudo-codice e che usa scope dinamico.
> Qual è l'output del programma?

```
int X;
X = 1;
int Y;
```

```
void fie() {
     foo();
     X = 0;
}
void foo() {
     int X;
     X = 5;
}
read(Y);
if(Y > 0) {
     int X;
     X = 4;
     fie(),
} else {
     fie();
}
write(X);
```

- Assumiamo `Y > 0` , `foo()` crea una sua `X` locale che sparisce quando termina la funzione, `fie()` inizializza a 0 la `X` creata all'interno dell'if, che però sparisce uscendo dall'istruzione condizionale. Dunque la `X` stampata è quella globale e vale 0.
- Assumiamo `Y <= 0`, `foo()` lavora sempre sulla sua variabile locale che poi scompare, mentre `fie()` inizializza a 0 la variabile globale. Dunque l'output è 0.

> ⚠ **Attenzione**
>
> In ML si usa lo scope statico!

05. Laboratorio 2

Esercizio 2.1

Scrivere una funzione che calcola il cubo di un numero reale.

```
> val cube = fn n:real => n * n * n;
(* val cube = fn: real -> real *)
> fun Cube (n: real) = n * n * n;
(* val Cube = fn: real -> real *)
> cube(2.9);
(* val it = 24.389: real *)
> Cube(2.9);
(* val it = 24.389: real *)
```

Esercizio 2.2

Scrivere una funzione `min3` che calcola il minore di una tupla di tre elementi del tipo `int*int*int`.

```
> fun min3 (a, b, c) =
    if a < b then
      if a < c then
        a
      else
        c
    else
      if c < b then
        c
      else
        b;
(* val min3 = fn: int * int * int -> int *)
> min3(13, 14, 15);
(* val it = 13: int *)
```

Esercizio 2.3

Scrivere una funzione `third` che calcola il terzo elemento di una lista (che ha almeno 3 elementi).

```
> fun third (l) = hd(tl(tl(l)));
(* val third = fn: 'a list -> 'a *)
> third [12, 13, 14, 15];
(* val it = 14: int *)
```

Il warning esce perché ML indica che la funzione potrebbe dare problemi con liste di meno di 3 elementi. Per il momento lo ignoriamo.

Esercizio 2.4

Scrivere una funzione `reverse` che inverte una tupla di lunghezza 3.

```
> fun reverse (a, b, c) = (c, b, a);
(* val reverse = fn: 'a * 'b * 'c -> 'c * 'b * 'a *)
> reverse (12, 13, 14);
(* val it = (14, 13, 12): int * int * int *)
```

Esercizio 2.5

Scrivere una funzione `thirdchar` che trova la stringa contenente il terzo carattere di una stringa data (la stringa ha almeno 3 caratteri).

```
> fun thirdchar (s:string) = str (hd(tl(tl(explode(s)))));
(* val thirdchar = fn: string -> string *)
> thirdchar ("ciao");
(* val it = "a": string *)
```

Esercizio 2.6

Scrivere una funzione `cycle` che shifta una lista di un elemento (la lista ha almeno un elemento).

```
> fun cycle (l) = tl(l @ [hd(l)]);
(* val cycle = fn: 'a list -> 'a list *)
> cycle [1, 2, 3, 4, 5, 6, 7];
(* val it = [2, 3, 4, 5, 6, 7, 1]: int list *)
```

Esercizio 2.7

Scrivere una funzione `min_max_pair` che, dati 3 interi (sotto forma di tupla), produca una coppia composta dall'elemento più piccolo e da quello più grande.

```
fun min3 (a, b, c) =
  if a < b then
    if a < c then a
    else c
  else
    if c < b then c
    else b;

fun max3 (a, b, c) =
  if a > b then
    if a > c then a
    else c
  else
    if c > b then c
    else b;

fun min_max_pair (a, b, c) = (min3(a, b, c), max3(a, b, c));

(* val min3 = fn: int * int * int -> int *)
(* val max3 = fn: int * int * int -> int *)
(* val min_max_pair = fn: int * int * int -> int * int *)

> min_max_pair (1, 2, 3);
(* val it = (1, 3): int * int *)
> min_max_pair (3, 4, 2);
(* val it = (2, 4): int * int *)
```

Esercizio 2.8

Scrivere una funzione `sort` che, dati 3 interi (sotto forma di tupla), produca una lista ordinata dei 3.

```
fun sort (a, b, c) =
  if a < b then
    if b < c then [a, b, c]
    else
      if a < c then [a, c, b]
      else [c, a, b]
  else
    if a < c then [b, a, c]
    else
```

```
      if b < c then [b, c, a]
      else [c, b, a];

(* val sort = fn: int * int * int -> int list *)

> sort (1, 2, 3);
(* val it = [1, 2, 3]: int list *)
> sort (1, 3, 2);
(* val it = [1, 2, 3]: int list *)
> sort (2, 1, 2);
```

Esercizio 2.9

Scrivere una funzione `rnd` che arrotonda un numero reale al numero decimale più vicino.

```
fun rnd (x: real) = real(round(x * 10.0)) / 10.0;
(* val rnd = fn: real -> real *)
> rnd (2.56);
(* val it = 2.6: real *)
> rnd (5.678);
(* val it = 5.7: real *)
```

Esercizio 2.10

Scrivere una funzione `rem` che, data una lista, rimuove il secondo elemento (la lista ha 2 o + elementi).

```
fun rem (l) = [hd(l)] @ tl(tl(l));
(* val rem = fn: 'a list -> 'a list *)
> rem [1,2,3,4];
(* val it = [1, 3, 4]: int list *)
> rem [1,2];
(* val it = [1]: int list *)
```

06. Astrazione dei controlli e ricorsione

Modalità di passaggio di parametri a funzione

Una funzione o procedura è un pezzo di codice identificato da un nome, con un environment locale ed in grado di scambiare informazioni con il resto del codice tramite dei parametri.

Meccanismi per lo scambio di informazioni con il codice esterno:

- parametri
- valori di ritorno
- environment non locale

I parametri possono essere di tre tipologie:

1. parametri di input
2. parametri di output
3. parametri di input/output

1. Passaggio per valore

Viene passata una copia del parametro (o dell'r-value passato alla funzione), che viene trattata come una variabile locale.
La trasmissione avviene dal main alla funzione.
Eventuali modifiche al parametro formale non toccano il parametro attuale e, una volta che la proceduta termina, il parametro formale viene eliminato.

```
int y = 1;
void foo(int x) {
     x = x+1;
}
...
y = 1;
foo(y+1);
```

Quando viene chiamata `foo()` gli viene passato il parametro `y+1`, che equivale a 2.
Viene quindi creata una copia di `x` e questa viene incrementata a 3. Successivamente la copia è distrutta.
Dunque, alla fine del programma, `x` non esiste e `y` è uguale a 1.

PRO:

- semantica più semplice
- accesso rapido al parametro formale

CONTRO:

- la chiamata può essere costosa se l'elemento da copiare è pesante

Passaggio per riferimento

Alla funzione viene passato un riferimento al parametro attuale. Quest'ultimo deve essere un'espressione con l-value (= un'espressione che identifica un'area di memoria ben definita).
Il riferimento al parametro e il parametro attuale puntano alla stessa cosa (aliasing). Quindi ogni modifica fatta sul riferimento viene applicata anche al parametro attuale.
La trasmissione avviene dal main alla funzione e anche viceversa.
Alla fine della funzione il riferimento tra parametro formale e parametro attuale viene cancellato.

```
int y = 1;
void foo(reference int x) {
     x = x+1;
}
...
y = 1;
foo(y);
```

Quando viene chiamata la funzione `foo()` viene creata `x`, ossia un riferimento alla cella di memoria di `y`, che contiene il valore 1. Tale valore viene incrementato di 1 nella funzione e questa modifica permane anche fuori dalla funzione.
Dunque, alla fine del programma, `x` non esiste e `y` è uguale a 2.

PRO:

- la chiamata è più efficiente, perché non viene copiato nulla

CONTRO:

- la semantica è più complicata
- l'accesso al parametro formale è leggermente più lento

Passaggio per costante

Il parametro non viene passato esplicitamente alla funzione ma è una costante (`const` in C++, `final` in Java), dunque è visibile nella funzione. Chiaramente è passato in sola lettura, non può essere modificato in nessun modo.
L'implementazione del passaggio è a discrezione del compilatore (i parametri più grossi solitamente sono passati per riferimento, mentre quelli piccoli per valore).

Passaggio per risultato

Il parametro attuale è un'espressione il cui risultato è un l-value. Non c'è nessun collegamento tra il parametro formale e il parametro attuale nel corpo della funzione. Quando la procedura termina, il valore del parametro formale è assegnato alla variabile corrispondente all'l-value del parametro attuale.
La trasmissione avviene output-only, ossia solo dalla procedura al main.

```
void foo(result int x) {
     x = 8;
}
...
y = 1;
foo(y);
```

In questo caso, quando la funzione `foo()` termina il valore di `x` viene assegnato alla variabile ottenuta tramite l'value di `y`.

Chiamata per valore risultato

Simile alla precedente, ma questa volta la trasmissione è bidirezionale, con il parametro formale che è usato come una variabile locale. Il parametro attuale deve essere un'espressione che produce un l-value.
All'inizio della funzione, il parametro attuale viene valutato e il suo r-value viene assegnato al parametro formale. Alla fine della funzione, il valore del parametro formale viene assegnato alla variabile corrispondente al parametro attuale.

```
void foo(value-result int x) {
     x = x+1;
}
...
y = 8;
foo(y);
```

In questo caso, quando viene chiamata `foo()`:

1. `x` viene creata come variabile locale
2. ad `x` viene assegnato il valore di `y` (8)
3. `x = x+1` (9)
4. ad `y` viene assegnato il valore di `x`
5. `x` viene distrutta

Dunque, alla fine del programma `x` non esiste e `y` è uguale a 9.

La comunicazione dunque è bidirezionale.

Chiamata per nome

Viene creata una copia del parametro attuale nel parametro formale.
Una chiamata alla funzione `f` equivale a eseguire il corpo di `f` sopo aver sostituito i parametri attuali al parametro formale. Ogni volta che il parametro formale appare, valutiamo di nuovo quello attuale.
I parametri in questione sono di input/output, la trasmissione è bidirezionale.
Non è così semplice come appare, bisogna saper gestire variabili con lo stesso nome.
Questo metodo di passaggio di parametri a funzione non viene più usato nella programmazione imperativa.

```
int z, w;
void fie(name int x) {
     int y;
     y = x+1;
}
...
z = 1;
w = 2;
fie(z+w);
```

In questo caso la variabile `x` viene definita come `z+w` e quindi quando si va ad eseguire `y = x+1` in verità si ha `y = z+w+1`.

```
int z;
void fie(name int x) {
     int y;
     x = x+1;
     y = 0;
}
...
z = 1;
fie(z);
```

In questo caso `x` è definita come `z` e quindi quando si va ad eseguire `x = x+1` in verità si sta eseguendo anche `z = z+1`. Dunque alla fine della funzione `z` vale 2.

> ⚠ **Problema**
>
> Se, all'interno della funzione, viene dichiarata una nuova variabile con lo stesso nome di una che definisce un parametro formale, come nell'esempio seguente, il risultato sarebbe incorretto.

```
int x = 0;
int foo(name int y) {
```

```
    int x = 2;
    return x + y;
}
...
int a = foo(x+1);
```

Il parametro attuale `x` viene "catturato" dalla dichiarazione locale e si ha che `x+y` diventa `x+(x+1)`, ma una riga prima `x` è stata ridefinita a 2.

Per evitare questa situazione, il parametro non deve essere valutato nell'ambiente della funzione, ma in quello del chiamante. La soluzione è di passare una coppia (*closure*) `<exp,env>` invece di passare soltanto `x+1`, con:

- `exp` che è il parametro attuale, non valutato
- `env` è l'environment al momento della chiamata
  In questo modo `y` ha memoria della `x` dichiarata nell'environment del chiamante e quindi si ha `x + y = 2 + 0 + 1 = 3`.

Esercizi

> ### Esercizio 5.1
>
> Dire cosa verrà stampato dal seguente frammento di codice che usa scoping dinamico. I parametri sono passati per riferimento.

```
{
    int x = 2;
    int fie(reference int y) {
        x = x+y;
    }
    {
        int x = 5;
        fie(x);
        write(x);
    }
    write(x);
}
```

Output: `10 2`

> ### Esercizio 5.2
>
> Dire cosa verrà stampato dal seguente frammento di codice che usa scoping dinamico. I parametri sono passati per nome.

```
{
     int x = 2;
     void fie(name int y) {
          x = x+y;
     }
     {
          int x = 5;
          {
               int x = 7;
          }
          fie(x++);
          write(x);
     }
     write(x);
}
```

Output: `6 7`

> **Esercizio 5.3**
>
> Dire cosa verrà stampato dal seguente frammento di codice. I parametri sono passati per valore risultato.

```
int X = 2;
void foo(value-result int Y) {
     Y++;
     write(X);
     Y++;
}
foo(X);
write(X);
```

Output: `2 4`

Ricorsione

> **Esempio**
>
> Scriviamo la funzione `reverse`, che inverte il contenuto di una lista.
>
> - Caso base: una lista vuota ritorna una lista vuota
> - Passo induttivo: concateniamo la `tail` invertita della lista alla e la `head` della lista

```
> fun reverse L =
    if L = null then nil
    else reverse (tl L) @ [hd L];
(* val reverse = fn: ''a list -> ''a list *)

> reverse [1,2,3,4];
(* val it = [4, 3, 2, 1]: int list *)
> reverse ["ab", "bc", "cd"];
(* val it = ["cd", "bc", "ab"]: string list *)
```

> ⚠ **Attenzione**
>
> Se, in ML, scriviamo una funzione usando la sintassi `val <nome> = fn <param> => <expression>`, nel caso in cui la funzione sia ricorsiva bisogna aggiungere la keyword `rec`.
> In questo caso diventerebbe, `val rec <nome> = fn <param> => <expression>`.
> Ciò non è più necessario quando definiamo la funzione con `fun`.

Ricorsione mutua

In ML, due funzioni possono chiamarsi a vicenda ricorsivamente.

> 💡 **Esempio**
>
> Scriviamo una funzione che prende una lista `L` e produce una lista che contiene solo gli elementi di `L` che si trovano a indice dispari (il primo, il terzo, ...).

Proviamo a scrivere tale funzione così:

```
> fun take(L) =
    if L = nil then nil
    else hd(L)::skip(tl(L));

> fun skip(L) =
    if L = nil then nil
    else tale(tl(L));

(* poly: : error: Value or constructor (skip) has not been declared
Found near if L = nil then nil else hd (L) :: skip (tl (L)) *)
```

Il primo tentativo non funziona, perché la `skip` viene usata prima di essere stata dichiarata. La soluzione è usare la keyword `and`, che ha sintassi come segue:

```
fun
     <definizione della prima funzione>
and
     <definizione della seconda funzione>
and
     ...
```

Riscriviamo dunque la soluzione all'esempio precedente usando `and`:

```
> fun
take(L) =
     if L = nil then nil
     else hd(L) :: skip(tl(L))
and
skip(L) =
     if L = nil then nil
     else take(tl(L));

(* val skip = fn: ''a list -> ''a list
val take = fn: ''a list -> ''a list *)

> take([1,2,3,4,5]);
(* val it = [1,3,5]: int list *)
```

07. Laboratorio 3 + Pattern Matching

Esercizio 3.1

Scrivere una funzione `fact` che calcola il fattoriale di `n` (> 1).

```
> fun fact (n) =
    if n = 0 then 1
    else fact(n-1)*n;
(* val fact = fn: int -> int *)
> fact(1);
(* val it = 1: int *)
> fact(10);
(* val it = 3628800: int *)
```

Esercizio 3.2

Scrivere una funzione `cyclei` che, dato un intero `i` e una lista `L = [a1, ..., an]`, cicla la lista `L` per `i` volte, ossia produce `[a_i+1, a_i+2, ..., a_n, a_1, ... a_i]`.

```
> fun cyclei (i, L) =
    if i = 0 then L
    else cycle(i-1, tl(L)@[hd(L)]);
(* val cyclei = fn: int * 'a list -> 'a list *)
> cyclei(2,[1,2,3,4]);
(* val it = [3, 4, 1, 2]: int list *)
> cyclei(4,["a","b","c","d","e"]);
(* val it = ["e", "a", "b", "c", "d"]: string list *)
```

Esercizio 3.3

Scrivere una funzione `duplicate` che duplica ogni elemento di una lista. Quindi, data la lista `L = [a_1, ..., a_n]`, restituisce `[a_1, a_1, a_2, a_2, ..., a_n, a_n]`.

```
> fun duplicate (L) =
    if L = nil then nil
```

```
    else [hd(L)] @ [hd(L)] @ duplicate(tl(L));
(* val duplicate = fn: ''a list -> ''a list *)
> duplicate([1,2,3,4]);
(* val it = [1, 1, 2, 2, 3, 3, 4, 4]: int list *)
> duplicate(["a","b","c"]);
(* val it = ["a", "a", "b", "b", "c", "c"]: string list *)
```

Esercizio 3.4

Scrivere una funzione `len` che calcola la lunghezza di una lista.

```
> fun len (L) =
    if L = nil then 0
    else len(tl(L))+1;
(* val len = fn: ''a list -> int *)
> len([1,2,3,4]);
(* val it = 4: int *)
> len(["a","b","c"]);
(* val it = 3: int *)
```

Esercizio 3.5

Scrivere una funzione `pow` che calcola x^i, dove `x` è un numero reale e `i` un intero non negativo.

```
> fun pow(x:real, i:int) =
    if i = 0 then 1.0
    else x*pow(x, i-1);
(* val pow = fn: real * int -> real *)
> pow(2.1, 3);
(* val it = 9.261: real *)
> pow(2.0, 3);
(* val it = 8.0: real *)
```

Esercizio 3.6

Scrivere una funzione `maxList` che calcola l'elemento maggiore di una lista di stringhe (in ordine lessicografico). La lista è non vuota.

```
> fun max(a: string, b: string) =
    if a > b then a
    else b;
```

```
> fun maxList(L: string list) =
    if L = nil then "a"
    else max(hd(L), maxList(tl(L)));

(* val max = fn: string * string -> string
val maxList = fn: string list -> string *)
> maxList(["a","abc","ab"]);
(* val it = "abc": string *)
```

Pattern - *Parte 1*

Sono una ~generalizzazione degli statement `case` o `switch` nei linguaggi procedurali. Sono molto potenti nella definizione delle funzioni.
La definizione delle funzioni usa una sequenza di pattern. Il primo che matcha l'argomento determina il valore ritornato.

```
fun <identifier> (<first pattern>) = <first expression>
  | <identifier> (<second pattern>) = <second expression>
  ...
  | <identifier> (<last pattern>) = <last expression>;
```

> 💡 **Esempio**
>
> Riscriviamo la funzione `reverse` per invertire una lista, con e senza il pattern matching.

- usando il pattern matching:

```
> fun reverse (nil) = nil
    | reverse (x::xs) = reverse(xs) @ [x];
(* val reverse = fn: 'a list -> 'a list *)
```

Il pattern `x::xs` matcha quando trova `elemento::lista`.

- senza il pattern matching:

```
> fun reverse L =
    if L = nil then nil
     else reberse tl(L) @ [hd(L)];
(* val reverse = fn: ''a list -> ''a list *)
```

> ⚠ **Attenzione**

La lista di alternative espressa nei pattern deve essere esaustiva (come un `if-then-else`). Altrimenti viene restituito un errore, come segue.

```
> fun reverse (nil) = nil;
(* poly: : warning: Matches are not exhaustive. Found near fun
reverse (nil) = nil
val reverse = fn: 'a list -> 'b list *)
reverse([3]);
(* poly: : warning: The type of (it) contains a free type variable.
Setting it to a unique monotype.
Exception- Match raised *)
```

Esercizio 3.7

Scrivere la funzione `fact` che calcola il fattoriale, usando i pattern.

```
> fun fact (0) = 1
    | fact (n) = n*fact(n-1);
(* val fact = fn: int -> int *)
> fact(1);
(* val it = 1: int *)
> fact(10);
(* val it = 3628800: int *)
```

Esercizio 3.8

Scrivere la funzione `cycle1` che cicla una lista di una posizione usando i pattern. Se la lista è vuota, ritornare la lista vuota.

```
> fun cycle1 (nil) = nil
    | cycle1 (L) = tl(L) @ [hd(L)];
(* val cycle1 = fn: 'a list -> 'a list *)
> cycle1([1,2,3,4,5]);
(* val it = [2, 3, 4, 5, 1]: int list *)
> cycle1([1]);
(* val it = [1]: int list *)
> cycle1(nil);
(* poly: : warning: The type of (it) contains a free type variable. //
Setting it to a unique monotype.
val it = []: _a list *)
```

Esercizio 3.9

Scrivere una funzione `cycleip` che, dato un intero `i` e una lista `L` = $[a_1, \ldots, a_n]$, cicla la lista per `i` volte, ossia produce $[a_i + 1, a_i + 2, \ldots, a_n, a_1, \ldots, a_i]$.

```
> fun cycleip (0, L) = L
    | cycleip (n, L) = cycleip(n-1, tl(L) @ [hd(L)]);
(* val cycleip = fn: int * 'a list -> 'a list *)
> cycleip(2,[1,2,3,4]);
(* val it = [3, 4, 1, 2]: int list *)
> cycleip(4,["a","b","c","d","e"]);
(* val it = ["e", "a", "b", "c", "d"]: string list *)
```

Esercizio 3.10

Scrivere una funzione `duplicatep` che duplica ogni elemento di una lista, usando i pattern.

```
> fun duplicatep (nil) = nil
    | duplicatep (L) = [hd(L)] @ [hd(L)] @ duplicatep(tl(L));
(* val duplicatep = fn: 'a list -> 'a list *)
> duplicatep([1,2,3,4]);
(* val it = [1, 1, 2, 2, 3, 3, 4, 4]: int list *)
> duplicatep([2.0]);
(* val it = [2.0, 2.0]: real list *)
```

Esercizio 3.11

Scrivere una funzione `power` che calcola x^i , dove `x` è un reale e `i` è un intero non negativo.

```
> fun power (n, 0) = 1.0
    | power (n, i) = n*power(n, i-1);
(* val power = fn: real * int -> real *)
> power(2.1,3);
(* val it = 9.261: real *)
> power(2.0,3);
(* val it = 8.0: real *)
```

Esercizio 3.12

> Scrivere una funzione `maxList` che calcola il massimo di una lista di reali, assumendo che la lista sia non vuota.

```
> fun max (a: real, b: real) =
    if a > b then a
    else b;

> fun maxList (nil) = 0.0
    | maxList (L: real list) = max(hd(L), maxList(tl(L)));

(* val max = fn: real * real -> real
val maxList = fn: real list -> real *)
> maxList([2.0]);
(* val it = 2.0: real *)
> maxList([2.0,5.1,4.2]);
(* val it = 5.1: real *)
```

08. Patterns e environment locali in ML

Pattern - *Parte 2*

La keyword `as`

È anche possibile assegnare un nome ai valori passati come argomenti dei pattern, in contemporanea alla loro valutazione, con la sintassi `<identifier> as <pattern>`

```
> fun reverse (nil) = nil
    | reverse (L as x::xs) = reverse(xs) @ [x];
(* val reverse = fn: 'a list -> 'a list *)
```

> **Esempio**
>
> Scriviamo una funzione `merge` che unisca due liste di interi `L` e `M`, assumendo che entrambe siano ordinate in modo crescente, in modo tale che anche la lista finale risulti ordinata.

- Caso base: se `L` è vuota, allora restituiamo `M` (e viceversa)
- Passo induttivo: confrontiamo la testa di `L` e la testa di `M`. Aggiungiamo quella maggiore alla lista finale e ricorsivamente chiamiamo la funzione sull'altra lista e la tail di quella usata.

```
> fun merge (nil,M) = M
    | merge (L,nil) = L
    | merge (L as x::xs, M as y::ys) =
        if x<y then x::merge(xs,M)
        else y::merge (L,ys);
(* val merge = fn: int list * int list -> int list *)
```

Variabile anonima (o wildcard)

Se vogliamo catturare una variabile ma non ci interessa usarla si può usare una wildcard, ossia nominarla con il simbolo `_`.

```
> fun comb (_,0) = 1
    | comb (n,k) =
        if k=n then 1
```

```
          else comb(n-1,k)+comb(n-1,k-1);
(* val comb = fn: int * int -> int *)
```

> ⚠ **Attenzione**
>
> Una variabile non può essere ripetuta in un pattern.

> ⚠ **Attenzione**
>
> Nei pattern non è consentito inserire:
>
> - operatori aritmetici
> - concatenazione di liste (`@`)
> - valori reali

Il pattern matching si può usare anche con la dichiarazione di funzione con `fn`.

```
> val day = fn 1 => "Monday"
             | 2 => "Tuesday"
             | _ => "Other";
(* val day = fn: int -> string *)
> day 5;
(* val it = "Other": string *)
```

Costrutto `Case`

È possibile fare pattern matching anche tramite il costrutto `case`, con la seguente sintassi:

```
fn x => case x of
    <pattern_1> => <expression_1>
    | <pattern_2> => <expression_2>
    ...
    | <pattern_n> => <expression_n>
```

Deve essere esaustivo, quindi deve contenere anche il caso di default.

Il `case`, di fatto, lavora come il pattern matching, ma a differenza di quest'ultimo può gestire anche i risultati di un'espressione.

Environment locale in ML

Per definire un environment locale si usa la seguente sintassi:

```
> fun name(par) =
    let
        val <first variable> = <first expression>;
        val <second variable> = <second expression>;
        …
        val <last variable> = <last expression>
    in
        <expression>
    end;
```

Dopo `let` si dichiarano le variabili locali e dopo `in` si possono valutare in espressioni.

> **Esempio**
>
> Scrivere una funzione che calcola la potenza centesima di un reale `x`

```
> fun hundredthPower (x:real) =
    let
        val four = x*x*x*x;
        val twenty = four * four * four * four * four
    in
        twenty * twenty * twenty * twenty * twenty
    end;
(* val hundredthPower = fn: real -> real *)
> hundredthPower 1.01;
(* val it = 2.704813829: real *)
> hundredthPower 2.0;
(* val it = 1.2676506E30: real *)
```

O anche, equivalentemente,

```
> fun hundredthPower (x:real) =
    let
        val x = x*x*x*x;
        val x = x*x*x*x*x;
    in
        x*x*x*x*x
    end;
(* val hundredthPower = fn: real -> real *)
```

> **Esempio**
>
> Scrivere una funzione `split(L)` che divide una lista `L` in 2 liste:
>
> - una contenente il primo elemento, il terzo, il quinto, ...

- una contenente il secondo, il quarto, ...

```
> fun split(nil) = (nil,nil)
    | split([a]) = ([a],nil)
    | split (a::b::cs) =
        let
            val (M,N) = split (cs)
        in
            (a::M,b::N)
        end;
(* val split = fn: 'a list -> 'a list * 'a list *)

> split [1,2,3,4,5];
(* val it = ([1,3,5], [2,4]): int list * int list *)
```

Esempio

Scrivere una funzione `merge` che esegue l'algoritmo di merge sort su una lista.

```
> fun merge (nil,M) = M
    | merge (L,nil) = L
    | merge (L as x::xs, M as y::ys) =
        if x<y then x::merge(xs,M)
        else y::merge (L,ys);
(* val merge = fn: int list * int list -> int list *)

> fun mergeSort (nil) = nil
    | mergeSort([a]) = [a]
    | mergeSort (L) =
        let
            val (M,N) = split(L);
            val M = mergeSort (M);
            val N = mergeSort (N)
        in
            merge (M,N)
        end;
(* val mergeSort = fn: int list -> int list *)
> mergeSort [1,4,2,3,8,7];
(* val it = [1, 2, 4, 3, 7, 8]: int list *)
> mergeSort([5,3,2,6,4,1]);
(* val it = [1, 2, 3, 4, 5, 6]: int list *)
```

09. Laboratorio 4

Esercizio 4.1

Scrivere una funzione `flip` che flippa gli elementi alternati, usando i pattern. Se la dimensione della lista è dispari, si lasci l'ultimo elemento invariato.
Quindi la lista $[a_1, a_2, \ldots, a_{n-1}, a_n]$ diventa $[a_2, a_1, \ldots, a_n, a_{n-1}]$.

```
> fun flip (nil) = (nil)
    | flip ([a]) = ([a])
    | flip (a::b::xs) = (b::a::flip(xs));
(* val flip = fn: 'a list -> 'a list *)
> flip [1,2,3,4,5];
(* val it = [2, 1, 4, 3, 5]: int list *)
> flip [1,2,3,4];
(* val it = [2, 1, 4, 3]: int list *)
```

Esercizio 4.2

Scrivere una funzione `remove` che, data una lista `L` e un intero `i`, ritorna `L` con l'`i`-esimo elemento eliminato. Se la lunghezza di `L` è minore di `i`, ritorna `L` invariata.

```
> fun remove (nil, i) = nil
    | remove (L, 1) = tl(L)
    | remove (L, i) = hd(L)::remove(tl(L), i-1);
(* val remove = fn: 'a list * int -> 'a list *)
> remove([1],1);
(* val it = []: int list *)
> remove([1,2,3],3);
(* val it = [1, 2]: int list *)
> remove([1,2],3);
(* val it = [1, 2]: int list *)
```

Esercizio 4.3

Scrivere una funzione `square` che prende in input un intero `n` e calcola il quadrato di `n` tramite la formula $n^2 = (n-1)^2 + 2n - 1$, usando i pattern.

```
> fun square (0) = (0)
    | square (n) = square(n-1)+2*n-1;
(* val square = fn: int -> int *)
> square(2);
(* val it = 4: int *)
> square(5);
(* val it = 25: int *)
> square(0);
(* val it = 0: int *)
```

Esercizio 4.4

Scrivere una funzione `flip` che prende in input una lista di coppie di interi e ordina ogni coppia in modo che il primo numero della coppia sia il minore.

```
> fun flip (nil) = (nil)
    | flip ((a,b)::xs) =
      if a < b then (a,b)::flip(xs)
      else (b,a)::flip(xs);
(* val flip = fn: (int * int) list -> (int * int) list *)
> flip [(1,2),(4,3)];
(* val it = [(1, 2), (3, 4)]: (int * int) list *)
> flip [(5,2),(4,3),(6,5),(1,2)];
(* val it = [(2, 5), (3, 4), (5, 6), (1, 2)]: (int * int) list *)
> flip [(1,1),(1,2)];
(* val it = [(1, 1), (1, 2)]: (int * int) list *)
> flip nil;
(* val it = []: (int * int) list *)
```

Esercizio 4.5

Scrivere una funzione `vowel` che prende in input una lista di caratteri e ritorna `true` se il primo elemento della lista è una vocale, usando i pattern.

```
> fun vowel (nil) = false
    | vowel ((#"a")::xs) = true
    | vowel ((#"e")::xs) = true
    | vowel ((#"i")::xs) = true
    | vowel ((#"o")::xs) = true
    | vowel ((#"u")::xs) = true
    | vowel (L) = false;
(* val vowel = fn: char list -> bool *)
> vowel [#"a"];
(* val it = true: bool *)
```

```
> vowel [#"b", #"c"];
(* val it = false: bool *)
> vowel [#"a", #"b"];
(* val it = true: bool *)
> vowel nil;
(* val it = false: bool *)
```

Esercizio 4.6

Assumiamo, per il momento, di rappresentare i set tramite liste: gli elementi sono ordinati in qualsiasi ordine, senza ripetizioni.
Scrivere una funzione `member(x,S)` per testare se `x` è un membro del set `S`, usando i pattern.

```
> fun member (x, nil) = false
    | member (x, a::xs) =
      if(x = a) then true
      else member(x, xs);
(* val member = fn: ''a * ''a list -> bool *)
> member(1, [2,3]);
(* val it = false: bool *)
> member(2, [2,3,1]);
(* val it = true: bool *)
> member(5,nil);
(* val it = false: bool *)
> member("b",["aa","c"]);
(* val it = false: bool *)
```

Esercizio 4.7

Scrivere una funzione `delete` che elimina l'elemento `x` dal set `S`, usando i pattern. Se `x` non appartiene al set `S`, la funzione ritorna `S` invariato.

```
> fun delete (x, nil) = (nil)
    | delete (x, a::xs) =
      if(x = a) then xs
      else a::delete(x, xs);
(* val delete = fn: ''a * ''a list -> ''a list *)
> delete(1,[2,3,4]);
(* val it = [2, 3, 4]: int list *)
> delete(1,[2,1,3]);
(* val it = [2, 3]: int list *)
> delete(1,nil);
(* val it = []: int list *)
```

```
> delete(#"a",[#"c",#"b",#"a"]);
(* val it = [#"c", #"b"]: char list *)
```

Esercizio 4.8

Scrivere una funzione `insert` che inserisce un elemento `x` nel set `S` (in qualsiasi posizione), usando i pattern. `x` viene inserita solo se non è già presente.

```
> fun insert (x, nil) = ([x])
    | insert (x, a::xs) =
      if(x = a) then a::xs
      else a::insert(x, xs);
(* val insert = fn: ''a * ''a list -> ''a list *)
> insert(2,[3,4,5]);
(* val it = [3, 4, 5, 2]: int list *)
> insert(3,[3,4,5]);
(* val it = [3, 4, 5]: int list *)
> insert(2, nil);
(* val it = [2]: int list *)
> insert (#"a",[#"b",#"c"]);
(* val it = [#"b", #"c", #"a"]: char list *)
```

Esercizio 4.9

Scrivere una funzione `insertAll` che prende un elemento `a` e una lista `L` e inserisce `a` come primo elemento di ogni elemento di `L`.

```
> fun insertAll (a, nil) = (nil)
    | insertAll (a, x::xs) = ([a]@x)::insertAll(a,xs);
(* val insertAll = fn: 'a * 'a list list -> 'a list list *)
> insertAll(1,[[2,3],[],[3]]);
(* val it = [[1, 2, 3], [1], [1, 3]]: int list list *)
> insertAll(1,nil);
(* val it = []: int list list *)
> insertAll(#"c",[[#"a",#"t"],[#"a",#"r"],nil]);
(* val it = [[#"c", #"a", #"t"], [#"c", #"a", #"r"], [#"c"]]: char list
list *)
```

Esercizio 4.10

Scrivere una funzione `powerSet` che prende una lista `L` e produce l'insieme delle parti di `L` (in qualsiasi ordine).

```
> fun insertAll (a, nil) = (nil)
    | insertAll (a, x::xs) = ([a]@x)::insertAll(a,xs);

> fun powerSet (nil) = ([[]])
    | powerSet (x::xs) = (powerSet(xs)@insertAll(x,powerSet(xs)));

(* val insertAll = fn: 'a * 'a list list -> 'a list list
val powerSet = fn: 'a list -> 'a list list *)
> powerSet([3]);
(* val it = [[], [3]]: int list list *)
> powerSet([1,2,3]);
(* val it = [[], [3], [2], [2, 3], [1], [1, 3], [1, 2], [1, 2, 3]]: int
list list *)
> powerSet nil;
(* poly: : warning: The type of (it) contains a free type variable.
Setting it to a unique monotype.
val it = [[]]: _a list list *)
> powerSet[#"a",#"c"];
(* val it = [[], [#"c"], [#"a"], [#"a", #"c"]]: char list list *)
```

Esercizio 4.11

Scrivere una funzione `prodDiff` che, data una lista di reali $[a_1, \ldots, a_n]$, calcoli $\prod_i (a_i - a)$.

```
> fun mulDif (x, nil) = 1.0
    | mulDif (x, a::xs) = (x-a)*mulDif(x,xs);

> fun prodDiff (nil) = 1.0
    | prodDiff ([x]: real list) = 1.0
    | prodDiff (x::xs) = mulDif(x, xs)*prodDiff(xs);

(* val mulDif = fn: real * real list -> real
val prodDiff = fn: real list -> real *)
> prodDiff([1.0,2.0,3.0]);
(* val it = ~2.0: real *)
> prodDiff(nil);
(* val it = 1.0: real *)
```

Esercizio 4.12

Scrivere una funzione `is_one` che ritorna `"one"` se il parametro passato è `1` e `"anything else"` altrimenti, usando il costrutto `case` e il pattern matching con `fun` e `fn`.

```
> fun is_one (1) = "one"
    | is_one (_) = "anything else";
(* val is_one = fn: int -> string *)
> is_one(1);
(* val it = "one": string *)
> is_one(3);
(* val it = "anything else": string *)
```

10. Input-output, eccezioni e funzioni polimorfe

Input e Output in ML

Dal momento che uno dei principi fondanti del paradigma di programmazione funzionale è di non avere side effects, input e output non dovrebbero essere ammessi. Nonostante ciò, ML permette di effettuarli.

Output

Il comando `print()` stampa una stringa.

```
> print;
(* val it = fn: string -> unit *)
```

> **Osservazione**
>
> Il tipo di `print` è `string -> unit`.
> `unit` è usato per espressioni e funzioni che non ritornano nulla ed ha un unico valore: `()`.

```
> print ("ab");
(* abval it = (): unit *)
> print ("ab\n");
(* ab
val it = (): unit *)
> fun testZero(0) = print("zero\n")
    | testZero(_) = print("not zero\n");
(* val testZero = fn: int -> unit *)
> testZero(2);
(* not zero
val it = (): unit *)
```

Per stampare un valore che ha tipo diverso da `string` è necessario usare funzioni di casting che trasformano altri tipi in `string`, permettendo di stamparli. Di seguito alcuni esempi:

- stampare un carattere:

```
> val c = #"a";
(* val c = #"a": char *)
```

```
> str;
(* val it = fn: char -> string *)
> print(str(c));
(* aval it = (): unit *)
```

- stampare un reale:

```
> val x = 1.0E50;
(* val x = 1E50: real *)
> print(Real.toString(x));
(* 1E50val it = (): unit *)
```

- stampare un intero:

```
> print(Int.toString(123));
(* 123val it = (): unit *)
```

- stampare un booleano:

```
> print(Bool.toString(true));
(* trueval it = (): unit *)
```

Compound statements

ML dà anche la possibilità di scrivere dei compound statements, che permettono di eseguire più comandi nella stessa linea di codice. Ritornano molto utili per le stampe.

```
> (print(Real.toString(1.0E50)); print(Int.toString(123)) );
(* 1E50123val it = (): unit *)
```

> **Osservazione**
>
> Il tipo di un compound statement è sempre quello dell'ultimo statement che contiene.

Input in ML

ML a disposizione numerose funzioni per la lettura in input da un file di testo. Di seguito sono riportate le principali:

(si assuma che il file in lettura sia come segue)

```
12
```

```
ab
```

- apertura di un file:

```
> val infile = TextIO.openIn("test");
(* val infile = ?: TextIO.instream  *)
```

- controllo se è stata raggiunta la fine dello stream:

```
> TextIO.endOfStream(infile);
(* val it = false: bool *)
```

- lettura di n caratteri (in questo caso 4):

```
> TextIO.inputN(infile,4);
(* val it = "12\na": string *)
> TextIO.inputN(infile,1);
(* val it = "b": string *)
> TextIO.inputN(infile,1);
(* val it = "\n": string *)
```

- lettura di una riga intera:

```
> val infile = TextIO.openIn("test");
(* val infile = ?: TextIO.instream *)
> TextIO.inputLine(infile);
(* val it = SOME "12\n": string option *)
> TextIO.inputLine(infile);
(* val it = SOME "ab\n": string option *)
> TextIO.inputLine(infile);
(* val it = NONE: string option *)
```

Osservazione

Il tipo di ritorno di queste ultime funzioni è **`string option`**. In generale, `T option` vale:

- `SOME`, se il valore da ritornare è di tipo `T`
- `NONE`, altrimenti

È utile perché permette, ad esempio, di accorgersi di quando siamo arrivati al termine dello stream.

- **chiusura** del file:

```
> TextIO.closeIn(infile);
(* val it = (): unit *)
```

- **lettura del file completo**:

```
> val infile = TextIO.openIn ("test");
(* val infile = ?: TextIO.instream *)
> val s = TextIO.input (infile);
(* val s = "12\nab\n": string *)
```

- **lettura** di un **singolo carattere**:

```
> TextIO.input1;
(* val it = fn: TextIO.instream -> char option *)
```

> **Osservazione**
>
> Queste funzioni di I/O possono essere usate anche all'interno di altre funzioni da noi definite.

> **Esercizio**
>
> Scrivere una funzione `makeList1` che legge i caratteri di un file e li mette in una lista.

```
> fun makeList1 (infile, NONE) = nil
    | makeList1 (infile, SOME c) = c::makeList1(infile,
TextIO.input1(infile));
(* val makeList1 = fn: TextIO.instream * char option -> char list *)
> fun makeList(infile) = makeList1(infile, TextIO.input1(infile));
(* val makeList = fn: TextIO.instream -> char list *)
```

- **lettura del prossimo carattere**, **senza modificare lo stream**:

```
> TextIO.lookahead;
(* val it = fn: TextIO.instream -> char option *)
```

- **controllo** se ci sono **almeno n caratteri rimanenti** nello stream (ritorna `SOME n` o `SOME m<n`)

```
> TextIO.canInput;
(* val it = fn: TextIO.instream * int -> int option *)
> TextIO.canInput(f,50);
(* val it = SOME 10: int option *)
```

Per trasformare un tipo `'a option` nel rispettivo tipo `'a` si usa la funzione `valOf`. Se però il valore è `NONE`, ritorna un'eccezione.

```
> valOf;
(* val it = fn: 'a option -> 'a *)
> fun convert (a: 'a option) = valOf(a);
(* val convert = fn: 'a option -> 'a *)
```

Eccezioni in ML

Esempi di **eccezioni predefinite** in ML:

```
> 5 div 0;
(* Exception- Div raised *)
> hd (nil: int list);
(* Exception- Empty raised *)
> tl (nil: real list);
(* Exception- Empty raised *)
> chr (500);
(* Exception- Chr raised *)
```

È possibile anche definire noi delle **eccezioni personalizzate**:

```
> exception Foo;
(* exception Foo *)
> Foo;
(* val it = Foo: exn *)
> raise Foo;
(* Exception- Foo raised *)
```

Osservazione

`exn` è il tipo delle eccezioni.

Esempio

Scriviamo una funzione che calcola $\binom{n}{m}$.

```
> exception BadN;
(* exception BadN *)
> exception BadM;
(* exception BadM *)
> fun comb(n,m)=
    if n<0 then raise BadN
    else if m<0 orelse m>n then raise BadM
    else if m=0 orelse m=n then 1
    else comb(n-1,m) + comb (n-1,m-1);
(* val comb = fn: int * int -> int *)
> comb(5,2);
(* val it = 10: int *)
> comb(~1,0);
(* Exception- BadN raised *)
> comb(5,6);
(* Exception- BadM raised *)
```

È possibile definire delle eccezioni che accettano dei tipi come argomento, con la seguente sintassi:

```
exception <identifier> of <type>
```

```
> exception Foo of string;
(* exception Foo of string *)
> Foo;
(* val it = fn: string -> exn *)
> raise Foo ("bar");
(* Exception- Foo "bar" raised *)
> raise Foo(5);
(* poly: : error: Type error in function application. *)
> raise Foo;
(* poly: : error: Exception to be raised must have type exn. *)
```

Tramite la sintassi

```
<expression> handle <match>
```

è possibile fare handling delle eccezioni.

```
> exception OutOfRange of int * int;

> fun comb1(n,m)=
    if n <= 0 then raise OutOfRange (n,m)
    else if m<0 orelse m>n then raise OutOfRange (n,m)
    else if m=0 orelse m=n then 1
```

```
    else comb1 (n-1,m) + comb1 (n-1,m-1);
(* val comb1 = fn: int * int -> int *)

> fun comb (n,m) = comb1 (n,m) handle
    OutOfRange (0,0) => 1
    | OutOfRange (n,m) => (
        print ("out of range: n=");
        print (Int.toString(n));
        print (" m=");
        print (Int.toString(m));
        print ("\n");
        0
    );
(* val comb = fn: int * int -> int *)
```

Polimorfismo delle funzioni in ML

Il polimorfismo di una funzione è la sua capacità di ricevere argomenti differenti in numero e/o tipo. Dal momento che ML è un linguaggio strongly typed, deve essere possibile determinare tutti i tipi prima ancora di fare il run del codice. Nonostante ciò, possiamo definire delle funzioni i cui tipi sono parzialmente o completamente flessibili.

> **Osservazione**
>
> Il tipo `'a` è usato per indicare il generico tipo polimorfico.

```
> fun identity (x) = x;
(* val identity = fn: 'a -> 'a *)
> identity (2);
(* val it = 2: int *)
> identity (2.0);
(* val it = 2.0: real *)
> identity (2) + floor (identity (3.5));
(* val it = 5: int *)
```

Operatori che limitano il polimorfismo:

- operazioni aritmetiche: `+`, `-`, `*`, `~`
- divisioni: `/`, `div`, `mod`
- operatori di disuguaglianza
- connettivi booleani: `andalso`, `orelse` e `not`
- operatori di concatenazione delle stringhe

- operatori di conversione di tipi: `ord`, `chr`, `real`, `str`, `floor`, `ceiling`, `round` e `truncate`

Operatori che permettono il polimorfismo:

- operatori delle tuple: `(..,..)`, `#1`, `#2`, ...
- operatori delle liste: `::`, `@`, `hd`, `tl`, `nil`, `[]`
- operatori di uguaglianza: `=`, `<>`

Uguaglianza di tipi in ML

I tipi che permettono l'uso di `=` e `<>` sono:gli interi, i booleani, i caratteri, tuple o liste, ma non i reali e le funzioni.

> **Osservazione**
>
> Le variabili di tipo che sono limitate a tipi che supportano gli operatori di uguaglianza vengono indicate con `''a`.

Esempi

```
> fun identity_eq(x) =
    if (x=x) then x
    else x;
(* val identity_eq = fn: ''a -> ''a *)
> identity_eq(2);
(* val it = 2: int *)
> identity_eq(2.0);
(* poly: : error: Type error in function application.
Function: identity_eq : ''a -> ''a
Argument: (2.0) : real
Reason: Can't unify ''a to real (Requires equality type)
Found near identity_eq (2.0)
Static Errors *)
```

```
> fun identity_t(x:''a) = x;
(* val identity_t = fn: ''a -> ''a *)
> identity_t(2);
(* val it = 2: int *)
> identity_t(2.0);
(* poly: : error: Type error in function application.
Function: identity_t : ''a -> ''a
Argument: (2.0) : real
Reason: Can't unify ''a to real (Requires equality type)
```

```
Found near identity_t (2.0)
Static Errors *)
```

```
> val M: ''a list=[];
(* val M = []: ''a list *)
> 2::M;
(* val it = [2]: int list *)
> val M: ''a list=[];
(* val M = []: ''a list *)
> 2.0::M;
(* poly: : error: Type error in function application.
Function: :: : real * real list -> real list
Argument: (2.0, M) : real * ''a list
Reason: Can't unify real to ''a (Requires equality type)
Found near 2.0 :: M
Static Errors *)
```

```
> fun first_eq(L) =
    if (hd(L)=hd(L)) then hd(L)
    else hd(L);
// val first_eq = fn: ''a list -> ''a *)
> first_eq([2]);
// val it = 2: int *)
> first_eq([2.0]);
(* poly: : error: Type error in function application.
Function: first_eq : ''a list -> ''a
Argument: ([2.0]) : real list
Reason: Can't unify ''a to real (Requires equality type)
Found near first_eq ([2.0])
Static Errors *)
```

Osservazione

Per controllare se una lista è vuota si può usare il comando `null(lista)`.

```
> fun rev3 (L) =
    if null(L) then nil
    else rev3(tl(L)) @ [hd(L)];
(* val rev3 = fn: 'a list -> 'a list *)
> rev3 [floor,trunc, ceil];
(* val it = [fn, fn, fn]: (real -> int) list *)
```

11. Laboratorio 5

Esercizio 5.1

Scrivere una funzione `is_lower_than5` che usa il costrutto `case` per simulare un `if-then-else` sul valore 5. Deve ritornare 1 se il valore in input è minore di 5, altrimenti ritorna 2.

```
> fun is_lower_than5 x = case x<5 of
      true => 1
    | false => 2;
(* val is_lower_than5 = fn: int -> int *)
> is_lower_than5(3);
(* val it = 1: int *)
> is_lower_than5(6);
(* val it = 2: int *)
> is_lower_than5(5);
(* val it = 2: int *)
```

Esercizio 5.2

Scrivere una funzione `thousandthPower` che, dato un numero reale `x` utilizza `let` `val` per calcolare x^{1000}.

```
> fun thousandthPower (x: real) =
  let
    val ten = x*x*x*x*x*x*x*x*x*x;
    val hun = ten*ten*ten*ten*ten*ten*ten*ten*ten*ten;
    val tho = hun*hun*hun*hun*hun*hun*hun*hun*hun*hun;
  in
    tho
  end;
(* val thousandthPower = fn: real -> real *)
> thousandthPower(1.0);
(* val it = 1.0: real *)
> thousandthPower(1.1);
(* val it = 2.469932918E41: real *)
```

Esercizio 5.3

Riscrivere la funzione `split` senza usare un pattern (la tupla) dentro a `let val`, ma usando le referenze alle componenti della tupla.

```
> fun split(nil) = (nil,nil)
    | split([a]) = ([a],nil)
    | split (a::b::cs) =
      let
        val M = #1 (split(cs));
        val N = #2 (split(cs));
      in
        (a::M,b::N)
       end;
(* val split = fn: 'a list -> 'a list * 'a list *)
> split [1,2,3,4,5,6];
(* val it = ([1, 3, 5], [2, 4, 6]): int list * int list *)
> split [1,2,3,4,5];
(* val it = ([1, 3, 5], [2, 4]): int list * int list *)
```

Esercizio 5.4

Riscrivere la funzione `powerSet` usando `let val` e calcolando `powerSet(xs)` una volta sola.

```
> fun insertAll (a, nil) = (nil)
    | insertAll (a, x::xs) = ([a]@x)::insertAll(a,xs);

> fun powerSet (nil) = ([[]])
    | powerSet (x::xs) =
      let
        val t = powerSet(xs);
      in
        (t@insertAll(x,t))
      end;

(* val insertAll = fn: 'a * 'a list list -> 'a list list
val powerSet = fn: 'a list -> 'a list list *)
> powerSet([1,2,3]);
(* val it = [[], [3], [2], [2, 3], [1], [1, 3], [1, 2], [1, 2, 3]]: int
list list *)
```

Esercizio 5.5

Scrivere una funzione `sumPairs` che prende una lista di coppie di interi e ritorna una coppia di interi, dove il primo elemento è la somma di tutti i primi elementi delle coppie e il secondo quella dei secondi elementi.

```
> fun sumPairs(nil) = (0,0)
    | sumPairs((x,y)::xs) =
      let
        val (M,N) = sumPairs(xs);
      in
        (x+M, y+N)
      end;
(* val sumPairs = fn: (int * int) list -> int * int *)
> sumPairs([(1,2),(3,4),(5,6)]);
(* val it = (9, 12): int * int *)
> sumPairs([]);
(* val it = (0, 0): int * int *)
```

Esercizio 5.6

Riscrivere la funzione `maxList` per trovare il massimo in una lista di reali, usando `let val`.

```
> fun maxList([x:real]) = x
    | maxList(x::zs) =
      let
        val mx = maxList(zs);
      in
        if x > mx then x
        else mx
      end;
(* Es6.sml:1: warning: Matches are not exhaustive.
val maxList = fn: real list -> real *)
> maxList([1.0,3.8,4.6,5.7,2.3,3.0,5.5]);
(* val it = 5.7: real *)
```

Esercizio 5.7

Scrivere una funzione `doubleExp` che, dato in input un reale `x` e un intero non negativo `i`, calcoli x^{2^i}, facendo una sola chiamata ricorsiva.

```
> fun doubleExp(x:real, 0) = (x)
    | doubleExp(x:real, i:int) =
```

```
      let
        val t = doubleExp(x, i-1);
      in
        t*t
      end;
(* val doubleExp = fn: real * int -> real *)
> doubleExp(1.1,3);
(* val it = 2.14358881: real *)
```

Esercizio 5.8

Scrivere una funzione `sumList` che prende una lista di interi e ritorna una coppia, in cui il primo elemento è la somma degli elementi in posizione pari e il secondo la somma di quelli in posizione dispari.

```
> fun sumList(nil) = (0,0)
    | sumList([a]) = (a,0)
    | sumList(a::b::xs) =
      let
        val (x, y) = sumList(xs);
      in
        (a+x,b+y)
      end;
(* val sumList = fn: int list -> int * int *)
> sumList([1,2,3,4]);
(* val it = (4, 6): int * int *)
> sumList([]);
(* val it = (0, 0): int * int *)
```

Esercizio 5.9

Scrivere una funzione `printList` che stampa una lista di interi data in input.

```
> fun printList(nil) = print("")
    | printList(x::xs) =
      (print(Int.toString(x)); print("\n"); printList(xs));
(* val printList = fn: int list -> unit *)
> printList [1,2,3];
(* 1
(* 2
(* 3
(* val it = (): unit *)
> printList nil;
(* val it = (): unit *)
```

Esercizio 5.10

Scrivere una funzione `comb: int*int -> unit` che calcola $\binom{n}{m}$ e stampa `m`, `n` e il risultato.

```
> fun factorial(0) = 1
    | factorial(n) = n*factorial(n-1);

> fun comb(n,m) = print("n is " ^ Int.toString(n) ^ "\nm is " ^
Int.toString(m) ^ "\nResult is " ^ Int.toString(factorial(n) div
(factorial(m)*factorial(n-m))) ^ "\n");

(* val factorial = fn: int -> int
val comb = fn: int * int -> unit *)
> comb(5,2);
(* n is 5
m is 2
Result is 10
val it = (): unit *)
```

Esercizio 5.11

Scrivere una funzione `printXs` che, dato un intero `n`, stampa il carattere `'X'` 2^N volte.

```
> fun printXs(0) = print("X")
    | printXs(n) = (printXs(n-1); printXs(n-1));
(* val printXs = fn: int -> unit *)
> printXs(3);
(* XXXXXXXXval it = (): unit *)
```

12. Funzioni di ordine superiore e curried

Alcuni linguaggi permettono di passare funzioni come argomenti a procedure o anche di ritornare funzioni come valore di ritorno delle procedure.

Funzioni come parametri di funzioni

La funzione viene passata come parametro ad un'altra funzione e poi viene chiamata tramite il parametro attuale.

```
{
     int x = 1;

     int f(int y){
          return x+y;
     }

     void g (int h(int b)){
          int x = 2;
          return h(3) + x;
     }
     ...
     {
          int x = 4;
          int z = g(f);
     }
}
```

La variabile `x` viene dichiarata tre volte. Quando chiamiamo `f`, quale `x` stiamo usando?

- se assumiamo di usare scope statico, allora usiamo la prima `x`, quella settata a 1
- se assumiamo di usare scope dinamico, allora andrebbe bene sia la seconda che la terza `x`, quindi dobbiamo specificare quale regola di binding viene applicata

Regole di binding

Quando una procedura viene passata come parametro, si crea una referenza tra un nome (parametro formale `h`) e una procedura (parametro attuale `f`). Bisogna specificare quale environment non-locale viene applicato quando `f` viene eseguita, che può essere:

- quello al momento in cui `f` viene passata come argomento (deep binding)

- quello al momento in cui `f` viene invocata (shallow binding)

Nell'esempio precedente con deep binding si userebbe la terza `x`, mentre con shallow binding la seconda.

> ⚠ **Attenzione**
>
> Con Static scope si usa sempre deep binding.

Funzioni come valore di ritorno di funzioni

La generazione di funzioni come valore di ritorno di una funzione permette la creazione dinamica di funzioni a runtime.

```
{
     int x = 1;

     void->int F () {
          int g () {
               return x+1;
          }
          return g;
     }

     void->int gg = F();
     int z = gg();
}
```

- `void -> int` denota il tipo della funzione (in questo caso non prende nessun argomento e ritorna un `int`)
- `void -> int F()` è la dichiarazione della funzione, che ritorna una funzione che non prende nessun argomento e ritorna un `int`
- `return g` ritorna la funzione, senza farne il run
- `gg` viene dinamicamente associata con il valore di ritorno di `F` (ossia `g`)
- la funzione `gg` ritorna un successore del parametro ricevuto in input

Funzioni di ordine superiore

> 💡 **Esempio**
>
> Supponiamo di voler scrivere una funzione che calcola l'approssimazione dell'integrale $\int_a^b f(x)dx$.

Quello che la funzione deve fare è dividere l'intervallo da `a` a `b` in `n` parti uguale e somma le aree degli `n` trapezi risultanti.

Definiamo la funzione `trap(a,b,n,F)`, dove `F` è la funzione da integrare e viene presa come parametro.

```
> fun trap (a,b,n,F) =
    if n<=0 orelse b-a<=0.0 then 0.0
    else
        let
            val delta = (b-a)/real(n)
        in
            delta * (F(a)+F(a+delta))/2.0 + trap (a+delta,b,n-1,F)
        end;
(* val trap = fn: real * real * int * (real -> real) -> real *)

> fun square(x:real) = x*x;
(* val square = fn: real -> real *)

> trap (0.0,1.0,8,square);
(* val it = 0.3359375: real *)
```

Esempio

Scriviamo una funzione `simpleMap` che prende una lista $[a_1, \dots a_n]$ e una funzione $f(x)$ in input e ritorna la lista $[f(a_1), \dots, f(a_n)]$.

```
> fun simpleMap (F,nil) = nil
    | simpleMap (F,x::xs) = F(x) :: simpleMap(F,xs);
(* val simpleMap = fn: ('a -> 'b) * 'a list -> 'b list *)
> simpleMap (square, [1.0,2.0,3.0]);
(* val it = [1.0, 4.0, 9.0]: real list *)
> simpleMap (~, [1,2,3]);
(* val it = [~1, ~2, ~3]: int list *)
> simpleMap ( fn x => x*x, [1.0,2.0,3.0]);
(* val it = [1.0, 4.0, 9.0]: real list *)
```

Attenzione

Non possiamo scrivere `fun f(x) = #1(x);` perché ML non è certo della grandezza della tupla. Non essendo sicuro, restituisce errore.

Esercizio

Scrivere una funzione `reduce` come segue:

- $[a_1]$ ritorna a_1
- $[a_1, \ldots, a_n]$: riduce la tail applicando una funzione F che prende una coppia e ritorna b e poi calcola $F(a_1, b)$, ossia $reduce([a_1, \ldots, a_n], F) = F(a_1, F(a_2, \ldots F(a_{n-1}, a_n)))$

Questa funzione è tale che se la funzione in input è la somma restituisce la somma di tutti gli elementi, se è l'operatore di prodotto ritorna il prodotto di tutti gli elementi, e così via...

```
> exception EmptyList;
(* exception EmptyList *)

> fun reduce (F,nil) = raise EmptyList
    | reduce (F,[a]) = a
    | reduce (F,x::xs) = F(x, reduce(F,xs));
(* val reduce = fn: ('a * 'a -> 'a) * 'a list -> 'a *)
```

Esercizio

Usare le funzioni `simpleMap` e `reduce` per calcolare la varianza di una lista di reali.

La varianza di una lista di reali è definita come

$$\mathrm{Var}(a_1, \ldots, a_n) = \frac{\sum_{i=1}^{n} a_i^2}{n} - \left(\frac{\sum_{i=1}^{n} a_i}{n} \right)^2$$

Definiamo due funzioni

```
> fun square (x:real) = x*x;
(* val square = fn: real -> real *)

> fun plus (x:real,y) = x+y;
(* val plus = fn: real * real -> real *)
```

La funzione per il calcolo della varianza risulta essere

```
> fun variance (L) =
    let
        val n = real(length(L))
    in
        reduce(plus,simpleMap(square,L))/n - square(reduce(plus,L)/n)
    end;
(* val variance = fn: real list -> real *)

> variance ([1.0,2.0,5.0,8.0]);
(* val it = 7.5: real *)
```

op

Usando `op` è possibile convertire un operatore infisso in un operatore prefisso.

```
> reduce (+, [1,2,3]);
(* poly: : warning: (+) has infix status but was not preceded by op. *)
> fun plus (x,y) = x+y;
(* val plus = fn: int * int -> int *)
> reduce(plus, [1,2,3]);
(* val it = 6: int *)

> reduce (op +, [1,2,3]);
(* val it = 6: int *)
```

> **Esercizio**
>
> Scrivere una funzione `filter` che prende come input una funzione booleana ed una lista e ritorna la lista contenente soltanto gli elementi che soddisfano la condizione della funzione booleana.

```
> fun filter (P,nil) = nil
    | filter (P,x::xs) =
        if P(x) then x::filter(P,xs)
        else filter (P,xs);
(* val filter = fn: ('a -> bool) * 'a list -> 'a list *)
> filter (fn x => x>=10, [1,10,23,5,16]);
(* val it = [10, 23, 16]: int list *)
```

Curried functions

Le funzioni, in ML, possono avere un solo argomento (e per questo si dicono unary). Quindi una funzione a due argomenti in realtà è:

- una funzione che prende come argomento una tupla

- oppure una funzione che restituisce un'altra funzione (currying)

> **Esempio**
>
> Scriviamo una funzione che calcola l'elevamento a potenza.

```
> fun exponent1 (x,0) = 1.0
    | exponent1 (x,y) = x * exponent1 (x,y-1);
(* val exponent1 = fn: real * int -> real *)
> exponent1 (3.0,4);
(* val it = 81.0: real *)
```

Questo è il primo modo possibile, classico.

```
> fun exponent2 x 0 = 1.0
    | exponent2 x y = x * exponent2 x (y-1);
(* val exponent2 = fn: real -> int -> real *)
> exponent2 3.0 4 ;
(* val it = 81.0: real *)
```

Questo è un altro modo, che fa uso del currying. Il significato del tipo della funzione (`real -> int -> real`) è in realtà `real -> (int -> real)`, cioè:

- prende un `real`
- restituisce una funzione `int -> real`

Quando scriviamo `exponent2 3.0 4` ML lo interpreta come `(exponent 3.0) 4`, dove `esponent2 3.0` restituisce una funzione, che viene poi applicata al parametro `4`.

Istanziazione parziale

La forma curried è utile perché permette di creare delle funzioni parzialmente istanziate (anche dette funzioni specializzate), dove solo alcuni argomenti sono dati.

```
> val g = exponent2 3.0;
(* val g = fn: int -> real *)
> g 4;
(* val it = 81.0: real *)
> g (4);
(* val it = 81.0: real *)
```

Nella prima riga abbiamo istanziato parzialmente la funzione `exponent2`. `g` è la funzione potenza con base `3.0`.

13. Laboratorio 6

Esercizio 6.1

Scrivere le espressioni per:

1. Aprire un file in lettura
2. Leggere 5 caratteri di tale file
3. Leggere una riga di tale file
4. Vedere qual è il prossimo carattere sullo stream, senza leggerlo
5. Leggere tutto il file intero
6. Chiudere il file

```
> val input = TextIO.openIn("Test.txt");

> TextIO.inputN(input,5);

> TextIO.inputLine(input);

> TextIO.lookahead(input);

> TextIO.input(input);

> TextIO.closeIn(input);
```

Esercizio 6.2

Scrivere una funzione `getList(filename)` che legge un file, estrae le parole e le mette in una lista.

```
> fun getWord(input: TextIO.instream) =
    if TextIO.endOfStream(input) then ""
    else
      let
        val c = TextIO.inputN(input, 1);
      in
        if (c = " " orelse c = "\n") then ""
        else c^getWord(input)
      end;
```

```
> fun getList(input: TextIO.instream) =
    if TextIO.endOfStream(input) then []
    else
      let
        val next = getWord(input);
      in
        next::getList(input)
      end;

(* val getWord = fn: TextIO.instream -> string
val getList = fn: TextIO.instream -> string list *)
> val input = TextIO.openIn("Test.txt");
(* val input = ?: TextIO.instream *)
> getList(input);
(* val it = ["Nel", "mezzo", "del", "cammin", "di", "nostra", "vita",
"mi", "ritrovai", "per", ...]: string list
val it = (): unit *)
```

Esercizio 6.3

Scrivere una funzione `returnThird(L)` che, data in input una lista di interi `L`, ritorna il terzo elemento. Se la lista è troppo corta, alza un'eccezione `shortList` e ne fa l'handling stampando "List too short.\n It only contains `n` elements.\n", dove `n` è il numero di elementi nella lista.

```
> exception shortList of int;

> fun listLength(nil) = 0
    | listLength(L: int list) = 1+listLength(tl(L));

> fun returnThird1(L) =
    let
      val size = listLength(L);
    in
      if size < 3 then raise shortList(size)
      else hd(tl(tl(L)))
    end;

> fun returnThird(L) = returnThird1(L) handle shortList(size) =>
(print("List too short.\n" ^ "It only contains " ^ Int.toString(size) ^ "
elements.\n"); 0);

(* exception shortList of int
val listLength = fn: int list -> int
val returnThird1 = fn: int list -> int
val returnThird = fn: int list -> int *)
> returnThird([1,2,3,4]);
```

```
(* val it = 3: int *)
> returnThird([1,2]);
(* List too short.
It only contains 2 elements.
val it = 0: int *)
```

Esercizio 6.4

Scrivere una funzione `fact` che, dato un intero `n`, ritorna n se n $0, 1$ se $n = 0$ e 0 se $n < 0$, con il messaggio di errore "Negative argument `x` found", dove `x` è l'argomento.

```
> exception negativeArgument of int

> fun fact1(0) = 1
    | fact1(n) =
      if n < 0 then raise negativeArgument(n)
      else n*fact1(n-1);

> fun fact(n) = fact1(n) handle negativeArgument(n) => (print("Negative
argument " ^ Int.toString(n) ^ " found\n"); 0);

(* val fact1 = fn: int -> int
exception negativeArgument of int
val fact = fn: int -> int *)
> fact(~2);
(* Negative argument ~2 found
val it = 0: int *)
> fact(0);
(* val it = 1: int *)
> fact(5);
(* val it = 120: int *)
```

Esercizio 6.5

Scrivere una funzione `tabulate` che prende un valore iniziale `a`, un incremento `delta`, un numero di punti `n` e una funzione `F` (dai reali ai reali) e stampa una tabella con 2 colonne, rispettivamente `x` e `F(x)`, con `x` che parte da `a` e viene incrementato di `delta` `n` volte.

```
> fun tabulate(a, delta, 0, f) = nil
    | tabulate(a, delta, n, f) = (print(Real.toString(a) ^ " " ^
Real.toString(f(a)) ^ "\n"); tabulate(a+delta, delta, n-1, f));
```

```
(* val tabulate = fn: real * real * int * (real -> real) -> 'a list *)
> tabulate(1.0,0.1,9,fn x => x*x);
(* poly: : warning: The type of (it) contains a free type variable.
Setting it to a unique monotype.
1.0 1.0
1.1 1.21
1.2 1.44
1.3 1.69
1.4 1.96
1.5 2.25
1.6 2.56
1.7 2.89
1.8 3.24
val it = []: _a list *)
```

Esercizio 6.6

Utilizzando la funzione `simpleMap`, rimpiazzare ogni elemento negativo di una lista di reali con 0.

```
> fun simpleMap (F,nil) = nil
    | simpleMap (F,x::xs) = F(x) :: simpleMap(F,xs);

> fun pos (a: real) =
    if a < 0.0 then 0.0
    else a

(* val simpleMap = fn: ('a -> 'b) * 'a list -> 'b list
val pos = fn: real -> real *)
> simpleMap(pos, [0.0,1.0,~2.1,~2.3]);
(* val it = [0.0, 1.0, 0.0, 0.0]: real list *)
```

Esercizio 6.7

Utilizzando la funzione `reduce`, trovare il massimo in una lista di reali.

```
> exception EmptyList;

> fun reduce(F, nil) = raise EmptyList
    | reduce(F, [a]) = a
    | reduce(F, x::xs) = F(x, reduce(F, xs));

> fun max(a: real, b: real) =
    if a > b then a
    else b;
```

```
(* exception EmptyList
val reduce = fn: ('a * 'a -> 'a) * 'a list -> 'a
val max = fn: real * real -> real *)
> reduce(max, [1.1,2.2,4.4,3.3]);
(* val it = 4.4: real *)
```

Esercizio 6.8

Utilizzando la funzione `filter`, trovare gli elementi in una lista di reali che sono maggiori di 0.

```
> fun filter(P, nil) = nil
    | filter(P, x::xs) =
      if P(x) then x::filter(P, xs)
      else filter(P, xs);

> fun pos(x: real) =
    if x < 0.0 then false
    else true;

(* val filter = fn: ('a -> bool) * 'a list -> 'a list
val pos = fn: real -> bool *)
> filter(pos, [1.1,~1.2,~1.3,1.4]);
(* val it = [1.1, 1.4]: real list *)
```

Esercizio 6.9

Scrivere una funzione `readAndSum` che, dato un file di testo contenente un numero per ogni riga, trasforma ogni riga nell'intero corrispondente e ritorna la somma di tutti questi.

```
> fun readAndSum1(input: TextIO.instream) =
    if TextIO.endOfStream(input) then 0
    else
      let
        val line = valOf(TextIO.inputLine(input));
        val x = valOf(Int.fromString(line));
      in
        x+readAndSum(input)
      end;

> fun readAndSum(file: string) =
    let
```

```
      val input = TextIO.openIn(file);
      val sum = readAndSum1(input);
    in
      TextIO.closeIn(input);
      sum
    end;

(* val readAndSum1 = fn: TextIO.instream -> int
val readAndSum = fn: string -> int *)
> readAndSum("numbers.txt");
(* val it = 23: int *)
```

Esercizio 6.10

Scrivere, nella forma curried, una funzione `applyList` che prende una lista di funzioni ed un valore e applica ogni funzione a tale valore, producendo una lista dei risultati.

```
> fun applyList nil a = nil
    | applyList (x::xs) a = (x(a)::applyList xs a);

> fun square (x) = x*x;

> fun neg (x) = ~x;

> fun zero (x) = 0;

> fun half (x) = x div 2;

(* val applyList = fn: ('a -> 'b) list -> 'a -> 'b list
val square = fn: int -> int
val neg = fn: int -> int
val zero = fn: 'a -> int
val half = fn: int -> int *)
> applyList([square, neg, zero, half]) 10;
(* val it = [100, ~10, 0, 5]: int list *)
```

14. Data types

Funzioni built-in di ordine superiore

> **Osservazione**
>
> In ML le funzioni built-in sono in forma curried.

Funzione `map`

Fa la stessa cosa della funzione `simpleMap`.
Accetta due parametri: una funzione e una lista di oggetti. Applica la funzione data ad ogni oggetto della lista e ritorna la lista dei risultati.

```
> map (fn x => x + 2) [1,2,3];
(* val it = [3, 4, 5]: int list *)
```

Definizione di `map`:

```
> fun map F =
    let
        fun M nil = nil
          | M(x::xs) = F x :: M xs
    in
        M
    end;
(* val map = fn: ('a -> 'b) -> 'a list -> 'b list *)
```

Funzione `foldr`

Fa la stessa cosa della funzione `reduce`, dall'elemento più a destra e procedendo verso sinistra.

```
> foldr (fn (a,b) => a+b) 2 [1,2,3]
(* val it = 8: int *)
```

Questo esempio fa $f(1, f(2, f(3, 2))) = 1 + (2 + (3 + 2))$.

Definizione di `foldr`:

```
> fun foldr F y nil = y
    | foldr F y (x::xs) = F (x,foldr F y xs);
(* val foldr = fn: ('a * 'b -> 'b) -> 'b -> 'a list -> 'b *)
```

Funzione `foldl`

Fa la stessa cosa della funzione `reduce`, dall'elemento più a sinistra e procedendo verso destra.

```
> foldl (fn (a,b) => a+b) 2 [1,2,3]
(* val it = 8: int *)
```

Questo esempio fa $f(3, f(2, f(1, 2))) = 3 + (2 + (1 + 2)))$.

Funzione `List.filter`

Processa una lista e ritorna una nuova lista contenente solo gli elementi che soddisfano un certo predicato. Prende in input una lista e un predicato.

```
> List.filter;
(* val filter : ('a -> bool) -> 'a list -> 'a list *)
> List.filter (fn x => x mod 2 = 0) [1, 2, 3, 4, 5];
(* val it = [2, 4]: int list *)
```

Composizione di funzioni in ML

La composizione di due funzioni F e G è la funzione H tale che $H(x) = G(F(x))$.

```
> fun comp (F,G,x) = G(F(x));
(* val comp = fn: ('a -> 'b) * ('b -> 'c) * 'a -> 'c *)
> comp (fn x=> x+3, fn y=>y*y+2*y, 10);
(* val it = 195: int *)
```

Operatore `o`

L'operatore `o` esegue la composizione di due funzioni. La prima indicata è quella più esterna nella composizione (nel caso precedente la G).

```
> fun F x = x+3;
(* val F = fn: int -> int *)
> fun G y = y*y + 2*y;
(* val G = fn: int -> int *)
> val H = G o F;
(* val H = fn: int -> int *)
> H 10;
(* val it = 195: int *)
```

```
> op o;
(* val it = fn: ('a -> 'b) * ('c -> 'a) -> 'c -> 'b *)
```

Data types

I Data types specificano dei valori e le operazioni consentite su tali valori.
In ML possiamo definire nuovi tipi o rinominare dei tipi già esistenti.

Rinominare tipi: `type`

Con la keyword `type` possiamo rinominare un tipo.

```
> type signal = int list;
(* type signal = int list *)
> val v = [1,2]: signal;
(* val v = [1, 2]: signal *)
```

È anche possibile parametrizzare una definizione di tipo. Come nel seguente esempio, dichiariamo `mapping` essere una lista di coppie di due tipi che però non vengono specificati (indicati con `'c` e `'d` ma corrispondono a `'a` e `'b`, in quanto ML da nomi ai tipi che ancora sono "generici" in ordine alfabetico).

```
> type ('c,'d) mapping = ('c * 'd) list;
(* type ('a, 'b) mapping = ('a * 'b) list *)
> val words = [("in",6),("a",1)] : (string,int) mapping;
(* val words = [("in", 6), ("a", 1)]: (string, int) mapping *)
```

Creare nuovi tipi: `datatype`

Con la keyword `datatype` è possibile creare dei nuovi tipi. Necessita di due parti:

1. Il costruttore del tipo (nome del datatype)
2. I costruttori dei possibili valori che può assumere

```
> datatype fruit = Apple | Pear | Grape;
(* datatype fruit = Apple | Grape | Pear *)
```

In questo esempio il datatype è `fruit` e i sui possibili valori sono `Apple`, `Pear`, `Grape`.

```
> fun isApple (x) = (x = Apple);
(* val isApple = fn: fruit -> bool *)
> isApple (Pear);
(* val it = false: bool *)
> isApple(Apple);
(* val it = true: bool *)
```

```
> isApple (Cherry);
(* poly: : error: Value or constructor (Cherry) has not been declared
Found near isApple (Cherry) *)
```

I costruttori dei possibili valori dei datatype possono prendere in input degli argomenti, secondo la seguente sintassi:

```
> datatype (<parameters>) <identifier> =
      <first constructor expression> |
      …
      <last constructor expression>
```

E il costruttore del datatype può essere parametrizzato:

```
> Cherry of int
```

In questo modo ogni espressione del tipo `Cherry(intero)` è accettata.

Il costruttore dei valori possibili viene usato per costruire delle espressioni che sono i valori per quel tipo.

Possiamo usare questi tipi per creare unioni di tipi diversi, ad esempio possiamo definire il tipo `element` che è una coppia `('a * 'b)` ma può anche essere un singolo `'a`:

```
> datatype ('a,'b) element =
    P of 'a * 'b |
    S of 'a;
(* datatype ('a, 'b) element = P of 'a * 'b | S of 'a *)
> P("a",1);
(* val it = P ("a", 1): (string, int) element *)
> P(1.0,2.0);
(* val it = P (1.0, 2.0): (real, real) element *)
> S(["a","b"]);
(* val it = S ["a", "b"]: (string list, 'a) element *)
```

> **Esempio**
>
> Data una lista di `elements`, scrivere una funzione `sumElList` che somma gli interi al secondo posto, quando esistono.

```
> fun sumElList (nil) = 0
    | sumElList (S(x)::L) = sumElList (L)
```

```
    | sumElList (P(x,y)::L) = y + sumElList (L);
(* val sumElList = fn: ('a, int) element list -> int *)
```

Datatypes definiti ricorsivamente

L'esempio tipico è quello dell'albero binario, in cui ogni nodo può:

- essere vuoto
- avere due figli, ognuno dei quali è a sua volta un nodo

```
> datatype 'label btree =
    Empty |
    Node of 'label * 'label btree * 'label btree;
(* datatype 'a btree = Empty | Node of 'a * 'a btree * 'a btree *)
```

```
> val myTree =
    Node("ML",
        Node("as",
            Node("a",Empty,Empty),
            Node("in",Empty,Empty)),
        Node("types",Empty,Empty)
    );

(* val myTree = Node ("ML", Node ("as", Node ("a", Empty, Empty), Node
("in", Empty, Empty)), Node ("types", Empty, Empty)): string btree *)
```

> **Esempio**
>
> Scrivere una funzione `printTree` che, data una `string btree`, stampa le label dell'albero: prima la label del primo nodo, poi quelle del suo sottoalbero di sinistra e poi quelle del suo sottoalbero di destra.

```
> fun printTree Empty = ()
    | printTree (Node(a,lt,rt)) = (print(a); print(" "); printTree(lt);
printTree(rt));
(* val printTree = fn: string btree -> unit *)
> printTree(myTree);
(* ML as a in types val it = (): unit *)
```

Ricorsione mutua nella definizione di datatype

Si utilizza la keyword `and`, come per le funzioni.

> **Esempio**

> Consideriamo i binary tree pari (ogni path dalla radice ad una fogli attraversa un numero pari di nodi) e i binary tree dispari. Proviamo a definirli.

```
> datatype 'label evenTree =
       Empty |
       Enode of 'label * 'label oddTree * 'label oddTree
  and
       'label oddTree = Onode of 'label * 'label evenTree * 'label
evenTree;
(* datatype 'a evenTree = Empty | Enode of 'a * 'a oddTree * 'a oddTree
datatype 'a oddTree = Onode of 'a * 'a evenTree * 'a evenTree *)
> val t1 = Onode (1,Empty,Empty);
(* val t1 = Onode (1, Empty, Empty): int oddTree *)
> val t2 = Onode (1,Empty,Empty);
(* val t2 = Onode (1, Empty, Empty): int oddTree *)
> val t3 = Enode (3,t1,t2);
(* val t3 = Enode (3, Onode (1, Empty, Empty), Onode (1, Empty, Empty)):
int evenTree *)
> val t4 = Onode (4,t3,Empty);
(* val t4 = Onode* 4, Enode (3, Onode (1, Empty, Empty), Onode (1, Empty,
Empty)), Empty): int oddTree *)
```

15. Laboratorio 7

Esercizio 7.1

Data una funzione `F`, che prende n parametri (con n fissato, in questo caso 3), definire una funzione curried che applica `F` e produce una funzione `G` tale che: $Gx_1 \dots x_n = F(x_1, \dots, x_n)$.

```
> fun curry f a b c = f(a,b,c);
(* val curry = fn: ('a * 'b * 'c -> 'd) -> 'a -> 'b -> 'c -> 'd *)
> curry(fn(x,y,z)=>x*y*z) 1 2 3;
(* val it = 6: int *)
```

Esercizio 7.2

Usare `map`, `foldr`, `foldl` per convertire una lista di interi in una lista di reali.

```
> fun toReal(l) = map (real) l;
(* val toReal = fn: int list -> real list *)
> toReal([1,2,3]);
(* val it = [1.0, 2.0, 3.0]: real list *)
```

Esercizio 7.3

Usare `map`, `foldr`, `foldl` per calcolare l'AND logico di una lista di booleani.

```
> fun andb(L: bool list) = foldr (fn(a,b) => (a andalso b)) true L;
(* val andb = fn: bool list -> bool *)
> andb [true, false, true];
(* val it = false: bool *)
```

Esercizio 7.4

Usare `map`, `foldr`, `foldl` per definire la funzione `implode`.

```
> fun implode(L) = foldl (fn(a: char, b: string) => (b^str(a))) "" L;
(* val implode = fn: char list -> string *)
> implode[#"b",#"c"];
(* val it = "bc": string *)
```

Esercizio 7.5

Dato un datatype `btree`, scrivere la funzione `postOrder` che ritorna la lista dei nodi di un albero binario visitato in post-order. Ossia la label di un nodo segue l'attraversamento post-order di entrambi i suoi sottoalberi.

```
> datatype 'a btree =
    Empty |
    Node of 'a * 'a btree * 'a btree;

> fun postOrder Empty = nil
    | postOrder(Node(v, lTree, rTree)) = postOrder(lTree) @
postOrder(rTree) @ [v];

(* datatype 'a btree = Empty | Node of 'a * 'a btree * 'a btree
val postOrder = fn: 'a btree -> 'a list *)

> postOrder (Node ("ML", Node ("as", Node ("a", Empty, Empty), Node ("in",
Empty, Empty)), Node ("types", Empty, Empty)));
(* val it = ["a", "in", "as", "types", "ML"]: string list *)
```

Esercizio 7.6

Dato il datatype `btree`, scrivere una funzione `inOrder` che ritorna la lista dei nodi di un albero binario visitato in in-order. Ossia la label di un nodo si trova tra l'attraversamento in-order del suo sottoalbero di sinistra e quello di destra.

```
> datatype 'a btree =
    Empty |
    Node of 'a * 'a btree * 'a btree;

> fun inOrder Empty = nil
    | inOrder(Node(v, lTree, rTree)) = inOrder(lTree) @ [v] @
inOrder(rTree);

> inOrder (Node ("ML", Node ("as", Node ("a", Empty, Empty), Node ("in",
Empty, Empty)), Node ("types", Empty, Empty)));
(* val it = ["a", "as", "in", "ML", "types"]: string list *)
```

Esercizio 7.7

Definire un tipo `mapTree` che è una specializzazione di `btree` tale per cui ha la label che è un insieme di coppie. Definire un albero `t1` che ha un singolo nodo e ha la coppia `("a", 1)` come label.

```
> datatype ('a, 'b) mapTree =
    Empty |
    Node of ('a * 'b) list * ('a, 'b) mapTree * ('a, 'b) mapTree;

> val t1 = Node([("a", 1)], Empty, Empty);

(* datatype ('a, 'b) mapTree =
    Empty
  | Node of ('a * 'b) list * ('a, 'b) mapTree * ('a, 'b) mapTree
val t1 = Node ([("a", 1)], Empty, Empty): (string, int) mapTree
val it = (): unit *)
```

Esercizio 7.8

Scrivere una funzione `sumTree` per un `mapTree` T di tipo `('a, 'b) mapTree`. La funzione visita l'albero e ritorna la somma della seconda componente di tutte le label dei nodi.

```
> datatype ('a, 'b) mapTree =
    Empty |
    Node of ('a * 'b) * ('a, 'b) mapTree * ('a, 'b) mapTree;

> fun sumTree Empty = 0
    | sumTree(Node((u,v), lTree, rTree)) = v + sumTree(lTree) +
sumTree(rTree);

(* datatype ('a, 'b) mapTree =
    Empty
  | Node of ('a * 'b) * ('a, 'b) mapTree * ('a, 'b) mapTree
val sumTree = fn: ('a, int) mapTree -> int
val it = (): unit *)

> sumTree (Node(("a",1), Node(("c",2), Empty, Node(("d",3), Empty,
Empty)), Empty));
(* val it = 6: int *)
```

Esercizio 7.9

Dato il datatype `btree`, definire una funzione `countInternalNodes` di tipo `('a -> bool) -> 'a btree -> int` che prende un predicato `p` (una funzione di tipo `'a -> bool`) e un `'a btree` e ritorna il numero di nodi interni (ossia nodi che hanno almeno un subtree non vuoto), il cuo valore soddisfa il predicato.

```
> datatype 'a btree =
    Empty |
    Node of 'a * 'a btree * 'a btree;

> fun countInternalNodes p Empty = 0
    | countInternalNodes p (Node(v, Empty, Empty)) = 0
    | countInternalNodes p (Node(v, lTree, rTree)) =
      if p(v) then 1+countInternalNodes p lTree + countInternalNodes p
rTree
      else countInternalNodes p lTree + countInternalNodes p  rTree;

(* datatype 'a btree = Empty | Node of 'a * 'a btree * 'a btree
val countInternalNodes = fn: ('a -> bool) -> 'a btree -> int *)

> countInternalNodes (fn x => x>5) (Node(10, Node(8, Empty, Empty),
Empty));
(* val it = 1: int *)
```

Esercizio 7.10

Dato il datatype `bltree`, definire una funzione `doubleTree` di tipo `int bltree -> int bltree` che prende un albero di interi e ritorna un nuovo albero dove ogni valore (sia sulle foglia che sui nodi) viene moltiplicato per 2.

```
> datatype 'a bltree =
      Empty
    | Leaf of 'a
    | Node of 'a * 'a bltree * 'a bltree;

> fun doubleTree Empty = Empty
    | doubleTree (Leaf(v)) = Leaf(2*v)
    | doubleTree (Node(v, lTree, rTree)) = Node(2*v, doubleTree(lTree),
doubleTree(rTree));

(* datatype 'a bltree = Empty | Leaf of 'a | Node of 'a * 'a bltree * 'a
bltree
val doubleTree = fn: int bltree -> int bltree *)
```

```
> doubleTree (Node(10, Leaf(5), Node(3, Empty, Leaf(1))));
(* val it = Node (20, Leaf 10, Node (6, Empty, Leaf 2)): int bltree *)
```

16. Data types astratti

Tipi di dati astratti

Quando definiamo una nuova struttura dati e non nascondiamo la sua specifica implementazione, noi potremmo tranquillamente lavorare sulla struttura interna del tipo di dato invece che utilizzare i metodi definiti apposta per interagire con essa.

Con l'astrazione dei dati si vuole **nascondere l'implementazione interna** del tipo di dato, che allo user deve essere invisibile. Si vuole **separare l'interfaccia dall'implementazione**.

> **Esempio**
>
> Supponiamo di avere un set. Con l'astrazione possiamo chiamare su di esso le funzioni dedicate (`empty`, `union`, `insert`, `is_member`), ma non conosciamo come è implementato il set (se è un vector, una lista o altro).

In ML (come in Java) esistono dei modi per "nascondere" l'implementazione di un tipo di dato e di renderne disponibile solo l'interfaccia: `signatures` e `structures`.

Structures e signatures

Le **`structure`** contengono una sequenza di dichiarazioni delle componenti di una struttura. Dentro una `structure` vengono definite tutte le variabili, le funzioni, le eccezioni, ... della struttura che si vuole creare.

Le **`signature`** sono la parte di "interfaccia" e contengono tutto quello che vogliamo che lo user possa vedere.

La relazione tra `structure` e `signature` è many-to-many.

> **Osservazione**
>
> Questo è il meccanismo che viene utilizzato anche quando noi chiamiamo i tipi `String`, `Int`, `Real`, ...
> Questi tipi, che noi consideriamo come "primitivi", sono però costruiti nello stesso esatto modo in cui noi possiamo costruire strutture (tipi) personalizzate.

`Structure`

Sintassi:

```
structure <identifier> = struct
    <elements of the structure>
end
```

Esempio:

```
> structure IntLT = struct
    type t = int
    val lt = (op <)
    val eq = (op =)
end;

(* structure IntLT:
sig val eq: ''a * ''a -> bool
    val lt: int * int -> bool
    eqtype t
end *)
```

In questo caso è stata creata una struttura `IntLT` che ha una variabile `t` di tipo `Int` e due funzioni `lt` e `eq` che implementano rispettivamente il controllo minore o uguale e il controllo uguale.

Esempio:

```
structure IntDiv = struct
    type t = int
    fun lt (m, n) = (n mod m = 0)
    val eq = (op =)
end;

(* Structure IntDiv:
sig val eq: ''a * ''a -> bool
    val lt: int * int -> bool
    eqtype t
end; *)
```

In questo esempio invece la struttura `IntDiv` ha una variabile intera `t` e implementa due funzioni `lt` e `eq` che controllato rispettivamente se un valore è divisibile per un altro dato e se due valori sono uguali.

> ⚠ **Attenzione**
>
> La funzione `lt` è introdotta dalla keyword `fun` perché si tratta di una definizione sintattica della funzione, ovvero stiamo definendo cosa fa quella funzione, che

parametri riceve in input e cosa dà in output.
La funzione `eq` invece è introdotta dalla keyword `val` perché di fatto stiamo assegnando l'operatore `=`, quindi un'operazione già fatta e finita. ad una funzione di nome `eq`. Stiamo facendo quindi un binding e non una definizione di funzione.

Long identifiers

I **long identifiers** servono per accedere alle funzioni di una struttura e, di conseguenza, usarle.

```
IntLT.lt (3,4);
(* val it = true: bool *)
IntDiv.lt(3,4);
(* val it = false: bool *)
IntLT.lt (3,4);
(* val it = true: bool *)
IntDiv.lt(3,4);
(* val it = false: bool *)
```

`Signature`

Le signatures vanno più nello specifico, specificando il tipo delle variabili e delle funzioni di una struttura.

Di fatto, la **signature** è come un **contratto che una structure deve rispettare**. Dice che una structure deve avere determinati tipi e funzioni, con determinati nomi e tipi.

Esempio:

```
signature ORDERED = sig
     type t
     val lt : t * t -> bool
     val eq : t * t -> bool
end;
```

Questo esempio ci dice che la signature `ORDERED` prevede un tipo `t` e due funzioni `lt` e `eq` con determinati tipi. Quindi tutte le structure che vogliono implementare tale signature devono avere quelle caratteristiche.

Esempio:

```
signature QUEUE = sig
     type 'a queue
     exception QueueError
     val empty : 'a queue
     val isEmpty : 'a queue -> bool
```

```
    val singleton : 'a -> 'a queue
    val insert : 'a * 'a queue -> 'a queue
    val remove : 'a queue -> 'a * 'a queue
end;
```

Esempio:

```
signature STACK =
sig
    val empty: 'a list
    val pop: 'a list -> 'a option
    val push: 'a * 'a list -> 'a list
    eqtype 'a stack
end;
```

In questo esempio ci viene detto che `'a stack` è equality type, ossia un tipo che supporta operazioni di equivalenza.

`structure` che implementa una `signature`

La signature è "l'interfaccia", dunque all'esterno è visibile soltanto quello che è scritto all'interno della signature. Nella structure possono esserci più cose ma a noi vengono nascoste se non fanno parte della signature.

```
structure Stack = struct
    type 'a stack = 'a list
    val empty = []
    val push = op::
    fun pop [] =NONE
    | pop (tos::rest) =SOME tos
end:> STACK;
```

> ⚠ **Attenzione**
>
> La dichiarazione `:>` dice che `Stack` è una implementazione della signature `STACK`. I componenti che non sono presenti nella signature non sono visibili all'esterno.

Alberi binari di ricerca (BST)

Proprietà fondamentale: se `x` è la label di un nodo `n`, allora ogni label `y` nel subtree di sinistra di `n` è tale che `lt(y,x)` è vero (ossia y <= x) e ogni label `y` nel subtree di destra di n è tale che `lt(x,y)` è vera (ossia x <= y).

Per i prossimi esempi avremo bisogno di una funzione `lt` che implementi l'ordinamento.

Esempi di funzioni di ordinamento per altri tipi:

```
> fun intLT (x,y) = x < y;
(* val intLT = fn: int * int -> bool *)

> fun lower (nil) = nil
    | lower (c::cs) = (Char.toLower c)::lower (cs);
(* val lower = fn: char list -> char list *)

> fun strLT (x,y) =
implode (lower (explode x)) < implode (lower (explode y));
(* val strLT = fn: string * string -> bool *)
```

Trovare un elemento in un BST

Scriviamo una funzione `Lookup` che va a cercare un valore all'interno del BST:

```
datatype 'a btree = Empty | Node of 'a * 'a btree * 'a btree

> fun lookup lt Empty x = false
    | lookup lt (Node(y,left,right)) x =
        if lt(x,y) then lookup lt left x
        else if lt(y,x) then lookup lt right x
             else true;
(* val lookup = fn: ('a * 'a -> bool) -> 'a btree -> 'a -> bool *)
```

Esempio:

```
> val t = Node ("ML",
    Node ("qas",
        Node ("a",Empty,Empty),
        Node ("in",Empty,Empty)
        ),
    Node ("types",Empty,Empty)
    );
(* val t = Node ("ML", Node ("as", Node ("a", Empty, Empty), Node ("in",
Empty,Empty)), Node ("types", Empty, Empty)): string btree *)
> lookup strLT t "function";
(* val it = false: bool *)
> lookup strLT t "ML";
(* val it = true: bool *)
```

Inserire un elemento in un BST

Scriviamo una funzione `insert` che inserisce un valore nel BST.

```
> fun insert lt Empty x = Node(x,Empty,Empty)
    | insert lt (T as Node (y,left,right)) x =
        if lt (x,y) then Node (y,(insert lt left x),right)
        else if lt (y,x) then Node (y,left,(insert lt right x))
            else T;
(* val insert = fn: ('a * 'a -> bool) -> 'a btree -> 'a -> 'a btree *)

> insert srtLT t "function";
(* val it = ("ML", Node ("as", Node ("a", Empty, Empty), Node ("in", Node
("function", Empty, Empty), Empty)), Node ("types", Empty, Empty)): string
btree *)
```

Rimuovere un elemento in un BST

Scriviamo una funzione `delete` che rimuove un elemento dal BST, se tale elemento esiste.
Come supporto, prima definiamo una funzione ausiliaria `deletemin` che, dati un BST, ritorna il BST dopo aver rimosso l'elemento con label minima (quindi quello più a sinistra).

```
> exception EmptyTree;
(* exception EmptyTree *)

> fun deletemin (Empty) = raise EmptyTree
    | deletemin (Node(y,Empty,right)) = (y,right)
    | deletemin (Node(w,left,right)) =
        let val (y,L) = deletemin(left)
        in (y,Node(w,L,right))
        end;
(* val deletemin = fn: 'a btree -> 'a * 'a btree *)

> fun delete lt Empty x = Empty
    | delete lt (Node(y,left,right)) x =
        if lt (x,y) then Node(y,(delete lt left x),right)
        else if lt (y,x) then Node(y,left,(delete lt right x))
            else
                case (left,right) of
                    (Empty,r) => r |
                    (l,Empty) => l |
                    (l,r) =>
                    let val (z,r1) = deletemin(r)
                    in Node (z,l,r1)
                    end;
(* val delete = fn: ('a * 'a -> bool) -> 'a btree -> 'a -> 'a btree *)
```

Visitare tutti i nodi di un BST

Scriviamo una funzione che somma tutti i valori del BST.

```
> fun sum (Empty) = 0
    | sum (Node(a,left,right)) = a + sum (left) + sum (right);
(* val sum = fn: int btree -> int *)
```

Scriviamo una funzione che crea una lista con i nodi dell'albero visitati in pre-order.

```
fun preOrder (Empty) = nil
    | preOrder(Node(a,left,right)) = [a] @ preOrder (left) @ preOrder
(right);
(* val preOrder = fn: 'a btree -> 'a list *)
```

Alberi

Dichiarazione

Datatype per descrivere un albero radicato generico, non per forza binario.

```
> datatype ('label) tree =
    Node of 'label * 'label tree list;
(* datatype 'a tree = Node of 'a * 'a tree list *)

> Node (1, [
    Node (2,nil),
    Node (3, [
        Node (4,nil),
        Node (5, [
            Node (7,nil)
        ]),
        Node (6,nil)
    ])
]);
(* val it =
Node (1, [Node (2, []), Node (3, [Node (4, []), Node (5, [Node (7,
[...])]), Node (6, [])])]): int tree *)
```

Somma dei valori dei nodi

Scriviamo una funzione `sum` che somma tutte le label dei nodi dell'albero.

```
> fun sum (Node(a,nil)) = a
    | sum (Node(a,t::ts)) = sum(t) + sum (Node(a,ts));
(* val sum = fn: int tree -> int *)
```

Possiamo farlo equivalentemente usando le funzioni di ordine superiore

```
> fun sum (Node(a,L)) = a + foldr (op +) 0 (map sum L);
(* val sum = fn: int tree -> int *)
```

Strutture restrittive

Consideriamo una struttura `Mapping`, che manipola coppie chiave-valore, così definita:

```
> structure Mapping = struct
    exception NotFound;
    val create = nil;
    fun lookup (d,nil) = raise NotFound
      | lookup (d,(e,r)::es) =
            if d=e then r
            else lookup (d,es);
    fun insert (d,r,nil) = [(d,r)]
      | insert (d,r,(e,s)::es) =
            if d=e then (d,r)::es
            else (e,s)::insert(d,r,es)
end;
```

Ora come ora è una struttura polimorfica, lo possiamo notare facendo

```
> Mapping.create;
(* val it = []: 'a list *)
```

Adesso restringiamo la struttura `Mapping` e la costringiamo ad assumere solo coppie `(string, int)`, tramite una signature `SIMAPPING`, che applichiamo ad una struttura `SiMapping` a cui assegniamo `Mapping`.

```
> signature SIMAPPING = sig
    val create : (string * int) list;
    val insert : string * int * (string * int) list -> (string * int)
list;
    val lookup : string * (string * int) list -> int
end;

> structure SiMapping : SIMAPPING = Mapping;
(* structure SiMapping: SIMAPPING *)
```

Accesso ai campi della struttura

Per accedere delle variabili e delle funzioni definite dentro alle strutture dobbiamo sempre ricordare di anteporre il nome della struttura ed un punto.

```
> val m = SiMapping.create;
(* val m = []: (string * int) list *)
> val m = SiMapping.insert ("in",6,m);
(* val m = [("in", 6)]: (string * int) list *)
> val m = SiMapping.insert ("a",1,m);
(* val m = [("in", 6), ("a", 1)]: (string * int) list *)
> SiMapping.lookup ("in",m);
(* val it = 6: int *)
```

Apertura di una struttura

Se invece volessimo accedere a variabili e funzioni di una struttura senza anteporre il nome una possibile soluzione sarebbe quella di "aprire" la struttura, ossia di portarla al livello del programma principale. Tutte le variabili e le funzioni definite nella struttura diventano quindi delle variabili e funzioni accettate dal linguaggio ML. Questa operazione di apertura si fa con la keyword `open`.

> ⚠ **Attenzione**
>
> Quando apriamo una struttura con `open` dobbiamo ricordare che tutto ciò che normalmente ha lo stesso nome di ciò che è stato definito dentro alla struttura in questione verrà rinominato. Questo potrebbe portare a situazioni di disagio non volute.

```
> 2 >1;
(* val it = true: bool *)

> open String;
(* val < = fn: string * string -> bool
val <= = fn: string * string -> bool
val > = fn: string * string -> bool
… *)

> 1 > 2;
(* poly: : error: Type error in function application.
Function: > : string * string -> bool
Argument: (1, 2) : int * int
Reason:
Can't unify int (*In Basis*) with string (*In Basis*)
(Different type constructors)
Found near 1 > 2
Static Errors *)
```

Funtori

Immaginiamo di avere una struttura `BST` che rappresenta un albero binario di tipo astratto.

```
> structure BST = struct
    exception EmptyTree;
    datatype 'a btree = Empty | Node of 'a * 'a btree * 'a btree;
    fun lookup lt Empty x = false
      | lookup lt (Node(y,left,right)) x =
            if lt(x,y) then lookup lt left x
            else
                if lt(y,x) then lookup lt right x
                else true;
    fun insert lt Empty x = Node(x,Empty,Empty)
      | insert lt (T as Node (y,left,right)) x =
            if lt (x,y) then Node (y,(insert lt left x),right)
            else
                if lt (y,x) then Node (y,left,(insert lt right x))
                else T;
    fun deletemin (Empty) = raise EmptyTree
      | deletemin (Node(y,Empty,right)) = (y,right)
      | deletemin (Node(w,left,right)) =
        let
            val (y,L) = deletemin(left)
        in
            (y,Node(w,L,right))
        end;
    fun delete lt Empty x = Empty
      | delete lt (Node(y,left,right)) x =
            if lt (x,y) then Node(y,(delete lt left x),right)
            else
                if lt (y,x) then Node(y,left,(delete lt right x))
                else
                    case (left,right) of (Empty,r) => r
                                       | (l,Empty) => l
                                       | (l,r) => let val (z,r1) =
deletemin(r)
                    in
                        Node (z,l,r1)
                    end;
end;
```

Ora vogliamo rendere questa struttura parametrica. In particolare, vogliamo definire uno `string BST`.
Per fare ciò dovremmo definire una funzione `lt` appropriata per fare i confronti e riscrivere totalmente la struttura.

```
> fun lower (nil) = nil
    | lower (c::cs) = (Char.toLower c)::lower (cs);
```

```
(* val lower = fn: char list -> char list *)

> fun lt (x,y) = implode (lower (explode x)) < implode (lower (explode
y));
(* val lt = fn: string * string -> bool *)
```

Se ora dichiarassimo nuovamente la struttura `BST` chiamandola `StringBST` otterremmo questo output:

```
(* structure StringBST:
    sig
        exception EmptyTree
        datatype 'a btree = Empty |
                            Node of 'a * 'a btree * 'a btree
        val delete: string btree -> string -> string btree
        val deletemin: 'a btree -> 'a * 'a btree
        val insert: string btree -> string -> string btree
        val lookup: string btree -> string -> bool
end *)
```

Possiamo notare che il tipo astratto `'a` rimane in alcuni posti.

Qui entra in gioco il concetto di funtore. Un funtore è una struttura che prende come argomento una struttura e ritorna una nuova struttura.

```
functor <identifier> (<structure name>: <signature>) = <structure
definition>
```

Riprendiamo l'esempio precedente, dell'albero binario. Un funtore può prendere in ingresso una struttura con un operatore `lt` e produrre una struttura che incorpora tale operatore. Si seguono i seguenti step:

1. Definiamo una signature `TOTALORDER` che va in input al funtore

Tale signature deve contenere l'operatore `lt`, per l'ordinamento.

```
> signature TOTALORDER = sig
    type element;
    val lt : element * element -> bool
end;

(* signature TOTALORDER = sig type element val lt: element *
element -> bool end *)
```

2. Definiamo un funtore `MakeBST` che riceve una struttura `S` con signature `TOTALORDER` e produce una struttura

```
functor MakeBST (Lt: TOTALORDER):
     sig
          type 'label btree
          exception EmptyTeee;
          val create : Lt.element btree;
          val lookup : Lt.element * Lt.element btree -> bool;
          val insert : Lt.element * Lt.element btree -> Lt.element btree;
          val deletemin : Lt.element btree -> Lt.element Lt.element btree;
          val delete : Lt.element * LT.element btree -> Lt.element btree
end
=
     struct
          open Lt;
          datatype 'label btree =
               Empty |
               Node of 'label * 'label btree * 'label btree;
          val create = Empty;
          val lookup (x, Empty) = ...
          val insert (x, Empty) = ...
          exception EmptyTree;
          fun deletemin (Empty) = ...
          fun delete (x,Empty) = ...
end;
```

3. Definiamo una structure `String` con signature `TOTALORDER` e con un operatore di confronto tra stringhe

```
structure String: TOTALORDER =
struct
     type element = string;
     fun lt (x,y) =
          let
               fun lower (nil) = nil
                 | lower (c::cs) = (Char.toLower c)::lower (cs);
          in
               implode (lower (explode (x))) < implode (lower( explode(y)))
          end;
end;
```

4. Applichiamo `MakeBST` alla structure `String` per produrre la structure desiderata

```
structure StringBST = MakeBST (String);
```

17. Laboratorio 8

Esercizio 8.1

Definire la signature `SET` che preveda:

- un parametro di tipo: `'a set`
- un valore per un set vuoto: `emptyset`
- un operatore per testare se un elemento fa parte del set o no: `isin`
- un operatore per aggiungere un elemento ad un set: `addin`
- un operatore per rimuovere un elemento da un set: `removefrom`

```
signature SET = sig
  type 'a set
  val emptyset : 'a set
  val isin : 'a * 'a set -> bool
  val addin : 'a * 'a set -> 'a set
  val removefrom : 'a * 'a set -> 'a set
end;

(*signature SET = sig
  type 'a set
  val emptyset : 'a set
  val isin : 'a * 'a set -> bool
  val addin : 'a * 'a set -> 'a set
  val removefrom : 'a * 'a set -> 'a set
end;*)
```

Esercizio 8.2

Con la signature `SET` data, aggiungere la definizione per la struttura `Set` che implementi tale signature tale structure.

```
signature SET = sig
  type 'a set
end;

structure Set = struct
  type 'a set = 'a list
end:>SET;
```

```
(*signature SET = sig type 'a set end
structure Set: SET
val it = (): unit*)
```

Esercizio 8.3

Con la signature `SET` data, aggiungere la definizione per la struttura `Set` che implementi tale signature e aggiungere dei casi di test.

```
signature SET = sig
  type 'a set
  val emptyset: 'a set
end;

structure Set = struct
  type 'a set = 'a list
  val emptyset = []
end:>SET;

(*signature SET = sig val emptyset: 'a set type 'a set end
structure Set: SET
val it = (): unit*)
```

Esercizio 8.4

Con la signature `SET` data, aggiungere la definizione per la struttura `Set` che implementi tale signature e aggiungere dei casi di test.

```
signature SET = sig
  type ''a set
  val emptyset: ''a set
  val isin: ''a -> ''a set -> bool
end;

structure Set = struct
  type ''a set = ''a list
  val emptyset = []
  fun isin y nil = false
    | isin y (x::xs) = if y = x then true else isin y xs
end:>SET;

val a = Set.emptyset;
val b = Set.isin 1 a;
```

```
(*signature SET =
  sig val emptyset: ''a set val isin: ''a -> ''a set -> bool type 'a set
end
structure Set: SET
val a = ?: ''a Set.set
val b = false: bool
val it = (): unit*)
```

Esercizio 8.5

Con la signature `SET` data, aggiungere la definizione per la struttura `Set` che implementi tale signature e aggiungere dei casi di test.

```
signature SET = sig
  type ''a set
  val emptyset: ''a set
  val isin: ''a -> ''a set -> bool
  val addin: ''a -> ''a set -> ''a set
end;

structure Set = struct
  type ''a set = ''a list
  val emptyset = []
  fun isin y nil = false
    | isin y (x::xs) = if y = x then true else isin y xs
  fun addin y s = if isin y s then s else y::s
end:>SET;

val a = Set.emptyset;
val b = Set.isin 1 a;
val c = Set.addin 1 a;
val d = Set.isin 1 c;

(*signature SET =
  sig
    val addin: ''a -> ''a set -> ''a set
    val emptyset: ''a set
    val isin: ''a -> ''a set -> bool
    type 'a set
  end
structure Set: SET
val a = ?: ''a Set.set
val b = false: bool
val c = ?: int Set.set
```

```
val d = true: bool
val it = (): unit*)
```

Esercizio 8.6

Con la signature `SET` data, aggiungere la definizione per la struttura `Set` che implementi tale signature e aggiungere dei casi di test.

```
signature SET = sig
  type ''a set
  val emptyset: ''a set
  val isin: ''a -> ''a set -> bool
  val addin: ''a -> ''a set -> ''a set
  val removefrom: ''a -> ''a set -> ''a set
end;

structure Set = struct
  type ''a set = ''a list
  val emptyset = []
  fun isin y nil = false
    | isin y (x::xs) = if y = x then true else isin y xs
  fun addin y s = if isin y s then s else y::s
  fun removefrom y nil = nil
    | removefrom y (x::xs) = if y = x then xs else (x::(removefrom y xs))
end:>SET;

val a = Set.emptyset;
val b = Set.isin 1 a;
val c = Set.addin 1 a;
val d = Set.addin 2 c;
val e = Set.isin 1 d;
val f = Set.removefrom 1 d;
val g = Set.isin 1 f;

(*signature SET =
  sig
    val addin: ''a -> ''a set -> ''a set
    val emptyset: ''a set
    val isin: ''a -> ''a set -> bool
    val removefrom: ''a -> ''a set -> ''a set
    type 'a set
  end
structure Set: SET
val a = ?: ''a Set.set
val b = false: bool
val c = ?: int Set.set
val d = ?: int Set.set
```

```
val e = true: bool
val f = ?: int Set.set
val g = false: bool
val it = (): unit*)
```

Esercizio 8.7

Con il datatype `'a T` dato, si definisca una signature `TREE` che preveda:

- una funzione per calcolare il numero di nodi nell'albero, escludendo le foglie: `countNodes`
- una funzione per calcolare la profondità dell'albero, escludendo le foglie: `depth`
- una funzione che calcola l'albero specchiato, ossia l'albero ottenuto invertendo left e right subtree di ogni nodo: `mirror`

```
datatype 'a T =
  Lf |
  Br of 'a * 'a T * 'a T

signature TREE = sig
  val countNodes: 'a T -> int
  val depth: 'a T -> int
  val mirror: 'a T -> 'a T
end;

(*datatype 'a T = Br of 'a * 'a T * 'a T | Lf
signature TREE =
  sig
    val countNodes: 'a T -> int
    val depth: 'a T -> int
    val mirror: 'a T -> 'a T
  end
val it = (): unit*)
```

Esercizio 8.8

Data la signature `TREE`, definire la struttura `Tree`. Aggiungere dei casi di test.

```
signature TREE = sig
  datatype 'a T =
    Lf |
    Br of 'a * 'a T * 'a T
  val countNodes: 'a T -> int
```

```
  val depth: 'a T -> int
  val mirror: 'a T -> 'a T
end;

structure Tree = struct
  datatype 'a T =
    Lf |
    Br of 'a * 'a T * 'a T
  fun countNodes Lf = 0
    | countNodes (Br(v, left, right)) = ((countNodes left) + (countNodes
right) + 1)
  fun depth Lf = 0
    | depth (Br(v, left, right)) =
      let
        val l = depth left
        val r = depth right
      in
        if l > r then l+1 else r+1
      end;
  fun mirror Lf = Lf
    | mirror (Br(v, left, right)) = Br(v, mirror right, mirror left)
end:>TREE;

val a = Tree.Br(1,
          Tree.Br(2, Tree.Lf, Tree.Lf),
          Tree.Br(3,
            Tree.Br(4, Tree.Lf, Tree.Lf),
            Tree.Br(5, Tree.Lf, Tree.Lf)));
val b = Tree.countNodes a;
val c = Tree.depth a;
val d = Tree.mirror a;

(*signature TREE =
  sig
    datatype 'a T = Br of 'a * 'a T * 'a T | Lf
    val countNodes: 'a T -> int
    val depth: 'a T -> int
    val mirror: 'a T -> 'a T
  end
structure Tree: TREE
val a = Br (1, Br (2, Lf, Lf), Br (3, Br (4, Lf, Lf), Br (5, Lf, Lf))):
   int Tree.T
val b = 5: int
val c = 3: int
val d = Br (1, Br (3, Br (5, Lf, Lf), Br (4, Lf, Lf)), Br (2, Lf, Lf)):
   int Tree.T
val it = (): unit*)
```

Esercizio 8.9

Dato il tipo `('a,'b) mapTree`, definito come un BST in cui l'ordine è definito dalla prima componente della tupla, scrivere una funzione `lookup lt T a` che cerca una coppia `(a, b)` la cui prima componente sia `a`. Se la trova, ritorna `b`, altrimenti ritorna l'eccezione `Missing`.
Scrivere un `(int, String) mapTree` per testare la funzione.

```
datatype 'a btree =
  Empty |
  Node of 'a * 'a btree * 'a btree;

type ('a, 'b) mapTree = ('a * 'b) btree;

val lt = (op <);

exception Missing;

fun lookup lt (Empty) (a, b) = raise Missing
  | lookup lt (Node((x, y), left, right)) (a, b) =
    if x = a then y
    else
      if lt(x, a) then lookup lt right (a, b)
      else lookup lt left (a, b);

val t : (int, string) mapTree =
  Node((5, "cinque"),
    Node((3, "tre"),
      Node((1, "uno"), Empty, Empty),
      Node((4, "quattro"), Empty, Empty)),
    Node((8, "otto"),
      Node((6, "sei"), Empty, Empty),
      Node((10, "dieci"), Empty, Empty))
  );

val test1 = lookup lt t (5, "");
val test2 = lookup lt t (1, "");
val test3 = lookup lt t (6, "");
val test4 = lookup lt t (10, "");

(*datatype 'a btree = Empty | Node of 'a * 'a btree * 'a btree
type ('a, 'b) mapTree = ('a * 'b) btree
val lt = fn: int * int -> bool
exception Missing
val lookup = fn: (''a * ''a -> bool) -> (''a * 'b) btree -> ''a * 'c -> 'b
val t =
   Node
    ((5, "cinque"),
```

```
     Node
      ((3, "tre"), Node ((1, "uno"), Empty, Empty),
       Node ((4, "quattro"), Empty, Empty)),
     Node
      ((8, "otto"), Node ((6, "sei"), Empty, Empty),
       Node ((10, "dieci"), Empty, Empty))): (int, string) mapTree
val test1 = "cinque": string
val test2 = "uno": string
val test3 = "sei": string
val test4 = "dieci": string
val it = (): unit*)
```

Esercizio 8.10

Dato il tipo `(a,b) mapTree` definito in precedenza, scrivere la funzione `assign lt T a b` che cerca nel BST la coppia `(a, c)`. Se la trova, sostituisce `c` con `b`, altrimenti inserisce la coppia `(a, b)` nel posto corretto nel BST.
Scrivere un `(int, String) mapTree` per testare la funzione.

```
datatype 'a btree =
  Empty |
  Node of 'a * 'a btree * 'a btree;

type ('a, 'b) mapTree = ('a * 'b) btree;

val lt = (op <);

fun assign lt (Empty) a b = (Node((a,b), Empty, Empty))
  | assign lt (Node((x,y), left, right)) a b =
    if x = a then (Node((x,b), left, right))
    else
      if lt(x, a) then (Node((x,y), left, assign lt right a b))
      else (Node((x,y), assign lt left a b, right))

val t : (int, string) mapTree =
  Node((5, "cinque"),
    Node((3, "tre"),
      Node((1, "uno"), Empty, Empty),
      Node((4, "quattro"), Empty, Empty)),
    Node((8, "otto"),
      Node((6, "sei"), Empty, Empty),
      Node((10, "dieci"), Empty, Empty))
  );

val test1 = assign lt t 1 "undici";
val test2 = assign lt t 7 "sette";
```

```
(*datatype 'a btree = Empty | Node of 'a * 'a btree * 'a btree
type ('a, 'b) mapTree = ('a * 'b) btree
val lt = fn: int * int -> bool
val assign = fn:
   (''a * ''a -> bool) -> (''a * 'b) btree -> ''a -> 'b -> (''a * 'b)
btree
val t =
   Node
    ((5, "cinque"),
     Node
      ((3, "tre"), Node ((1, "uno"), Empty, Empty),
       Node ((4, "quattro"), Empty, Empty)),
     Node
      ((8, "otto"), Node ((6, "sei"), Empty, Empty),
       Node ((10, "dieci"), Empty, Empty))): (int, string) mapTree
val test1 =
   Node
    ((5, "cinque"),
     Node
      ((3, "tre"), Node ((1, "undici"), Empty, Empty),
       Node ((4, "quattro"), Empty, Empty)),
     Node
      ((8, "otto"), Node ((6, "sei"), Empty, Empty),
       Node ((10, "dieci"), Empty, Empty))): (int * string) btree
val test2 =
   Node
    ((5, "cinque"),
     Node
      ((3, "tre"), Node ((1, "uno"), Empty, Empty),
       Node ((4, "quattro"), Empty, Empty)),
     Node
      ((8, "otto"),
       Node ((6, "sei"), Empty, Node ((7, "sette"), Empty, Empty)),
       Node ((10, "dieci"), Empty, Empty))): (int * string) btree
val it = (): unit*)
```

18. Lambda calcolo

Il Lambda calcolo è un linguaggio di programmazione molto semplice e Turing completo. È il primo linguaggio ed è stato creato, addirittura prima del concetto di linguaggio di programmazione stesso.

Il suo scopo è di porre delle basi matematiche formali per la programmazione funzionale.

Breve storia del lambda-calcolo

Il lambda-calcolo viene sviluppato dal logico Alonzo Church nel 1932, stesso periodo in cui Alan Turing stava inventando la macchina di Turing in Inghilterra. Turing incontra e studia da Church a Princeton e dimostra che il lambda-calcolo, la macchina di Turing e la ricorsione generale sono equivalenti e definiscono la classe delle funzioni computabili.

Il lambda-calcolo si chiama così perché inizialmente nei primi scritti di Church la variabile x veniva utilizzata per identificare la variabile di una funzione. In particolare, la lettera veniva preceduta dall'accento circonflesso. Per trovare un simbolo più comodo però fu scelto il simbolo λ.

Obiettivo

Porre le basi matematiche per la programmazione funzionale.

Se la macchina di Turing è il fondamento matematico per i linguaggi di programmazione imperativa, il lambda calcolo è la base teorica per i linguaggi di programmazione funzionale.

Ingredienti del λ-calcolo

1. variabili
2. funzioni, che sono delle lambda astrazioni sulle variabili
3. applicazione delle funzioni ai valori

> **Esempio**
>
> $(\lambda x.\, x^2)$ corrisponde alla funzione di elevamento al quadrato.
> $(\lambda x.\, x^2)7 = 49$ è l'applicazione di tale funzione ad un'espressione.

Un'espressione nel λ-calcolo può essere:

- x, una variabile $\in V$
- $\lambda x.\, e$, una funzione che prende in input un parametro x e valuta l'espressione e
- $e_1\ e_2$, l'applicazione di due espressioni

Operazioni del λ-calcolo

- Astrazione di un termine rispetto alla variabile x.
  $\lambda x.\, e$ è la funzione che, quando applicata ad un valore v, produce e in cui v sostituisce x.
- Applicazione di una funzione ad un argomento.
  $e_1\ e_2$ è l'applicazione di una funzione e_1 ad un argomento e_2.

Terminologia

Nella scrittura $\lambda x.\, e$ abbiamo che:

- λx si chiama binder
- e si chiama scope della funzione x

Se abbiamo $\lambda x.\, xy$ invece:

- λx è il binder
- x è una variabile bound
- y è una variabile free

Variabili free

L'insieme delle variabili free di un'espressione è definito (a seconda del tipo di espressione) come:

- $F_v(x) = x$
- $F_v(\lambda x.\, e) = F_v(e) \backslash x$
- $F_v(e_1 e_2) = F_v(e_1) \cup F_v(e_2)$

> **Esempio**
>
> Le variabili free dell'espressione $\lambda x.\, y(\lambda y.\, xyu)$ sono la prima y e u.

Variabili bound

L'insieme delle variabili bound di una espressione è definito (a seconda del tipo di espressione) come:

- $B_v = \emptyset$
- $B_v(\lambda x.\, e) = x \cup B_v(e)$

- $B_v(e_1e_2) = B_v(e_1) \cup B_v(e_2)$

> **Esempio**
>
> Le variabili bound dell'espressione $\lambda x.\, y(\lambda y.\, xyu)$ sono x e la seconda y.

Convenzioni

- L'associatività di una applicazione è alla sua sinistra.
  Ad esempio, yzx corrisponde a $(yz)x$
- Le parentesi possono essere utilizzate per leggibilità, ma non sono necessarie
- Il corpo della lambda si estende il più a destra possibile.
  Ad esempio, $\lambda x.\, x\lambda z.\, xzx$ corrisponde a $\lambda x.\, (x\lambda z.\, (xzx))$

> **Esercizio**
>
> Rendere le parentesi esplicite nella seguente espressione:
> $(\lambda p.\, pz)\lambda q.\, w\lambda w.\, wqzp$
>
> $(\lambda p.\, (pz))\lambda q.\, (w\lambda(((w.\, wq)z)p)))$

> **Esercizio**
>
> Nella seguente espressione dire quali sono, se ce ne sono, le variabili bound e a quali λ sono associate, e quali sono free: $\lambda s.\, sz\lambda q.\, sq$
>
> - la prima s è bound al λs
> - la z è free
> - la seconda s è bound al λs
> - la q è bound al λq

> **Esercizio**
>
> Nella seguente espressione dire quali sono, se ce ne sono, le variabili bound e a quali λ sono associate, e quali sono free: $(\lambda s.\, sz)\lambda q.\, w\lambda w.\, wqzs$
>
> - la prima s è bound al λs
> - la prima z è free
> - la prima w è free
> - la seconda w è bound al λw
> - la q è bound al λq
> - la seconda z è free

- la seconda s è free

Riduzione-β

La riduzione-λ non è altro che il processo che serve per ridurre le espressioni-λ, valutandole.

> **Esempi di riduzione-β**
>
> - $(\lambda x.\, x + x)5 \to 5 + 5 \to 10$
> - $(\lambda x.3)5 \to 3$
> - $(\lambda x.\, (\lambda y.\, y - x))4\ 5 \to (\lambda y.\, y - 4)5 \to 5 - 4 \to 1$
> - $(\lambda f.\, f\ 3)(\lambda x.\, x + 1) \to (\lambda x.\, x + 1)3 \to 4$

> **Esempi generici di riduzione-β**
>
> - $(\lambda x.\, x)z \to z$
> - $(\lambda x.\, y)z \to y$
> - $(\lambda x.\, x\ y)z \to z\ y$
> - $(\lambda x, x\ y)(\lambda z.\, z) \to (\lambda z.\, z)y \to y$
> - $(\lambda x.\, \lambda y.\, x\ y)z \to \lambda y.\, z\ y$

Come abbiamo visto dagli esempi precedenti, la riduzione-β può essere applicata alle funzioni di ordine superiore. Consideriamo, per esempio, una funzione che data una funzione f restituisca la funzione $f \circ f$: $\lambda f.\, \lambda x.\, f(fx)$
Per verificare se funziona, applichiamola alla funzione successore, definita come $\lambda y.\, y + 1$:
$(\lambda f.\, \lambda x.\, f(fx))(\lambda y.\, y + 1) \to$
$\lambda x.\, (\lambda y.\, y + 1)((\lambda y.\, y + 1)x) \to$
$\lambda x.\, (\lambda y.\, y + 1)(x + 1) \to$
$\lambda x.\, (x + 1) + 1$
Ed effettivamente β-ridurre l'espressione ci porta ad ottenere il successivo del successivo.

Sostituzione

Sostituzione, nel caso di un valore:
$(\lambda x.\, e_1)e_2 \mapsto e_1[e_2/x]$ diventa

- e_2, se $e_1 = x$
- e_1, se $e_1 \neq x$

Sostituzione, nel caso di un'applicazione
$(\lambda x.\, e_1)e_2 \mapsto e_1[e_2/x]$ diventa $(e_{11}e_{12})[e_2/x] = (e_{11}[e_2/x]e_{12}[e_2/x])$

Sostituzione, nel caso di un'astrazione
$(\lambda x.\, e_1)e_2 \mapsto e_1[e_2/x]$, con $e_1 = \lambda y.\, e$, diventa:

- $(\lambda y.\, e)[e_2/x] = \lambda y.\, e[e_2/x]$
- $(\lambda y.\, e)[e_2/x] = \lambda y.\, e$

α-equivalenza e α-conversione

Cosa succede, invece, se abbiamo $y \in F_v(e_2)$?
Ad esempio in $(\lambda x.\, \lambda y.\, xy)y$, otterremmo:
$(\lambda x.\, \lambda y.\, xy)y \mapsto (\lambda x.\, \lambda yyy)$, ma ciò è un errore perché la seconda y è prima una variabile libera e poi diventa una variabile bound. Errore!
La soluzione è rinominare la prima y (quella dentro la λ). Rinominiamola, ad esempio, v.
In questo modo otteniamo $(\lambda x.\, \lambda y.\, xy)y \mapsto (\lambda x.\, \lambda vyv)$

Due espressioni e_1 e e_2 si dicono equivalenti quando differiscono solo nei nomi delle variabili bound.
Ad esempio, se y non è presente nell'espressione e, allora $\lambda x.\, e = \lambda y.\, e[y/x]$.

L'α-conversione viene usata proprio per evitare il problema di cui abbiamo parlato prima.

Regole per la riduzione β

1. Le applicazioni sono associative verso sinistra: $MNL = (MN)L$
2. Il corpo di una λ si estende verso destra il più possibile: $\lambda x.\, x\lambda z.\, xzx$ corrisponde a $(\lambda x.\, (x\lambda z.\, (xzx)))$
3. Se ci sono le parentesi, utilizzare le regole di precedenza dettate da esse
4. Altrimenti, la precedenza è data a sinistra
5. Attenzione a quando una variabile viene catturata, in particolare quando una variabile free diventa bound. Ciò genera un errore e si deve procedere a rinominare tale variabile.

> ⚠ **Attenzione**
>
> Le riduzioni β non sono simmetriche.

> 👁 **Osservazione**
>
> Un'espressione si dice in forma normale se non si può β-ridurre ulteriormente.

Una riduzione-β può terminare in una forma normale o può anche andare avanti all'infinito, come nell'esempio $(\lambda x.\, xx)(\lambda x.\, xx)$

> **Teorema**
>
> Se e può essere ridotta a e_1 grazie ad una riduzione-β e e può essere ridotta a e_2 grazie ad una riduzione-β, allora esiste un e_3 tale che sia e_1 che e_2 possono essere ridotti a e_3 tramite una riduzione-β.

> **Esercizio**
>
> Ridurre alla forma normale l'espressione $(\lambda x.\, x(xy))(\lambda z.\, zx)$.

Soluzione:
$(\lambda x.\, x(xy))(\lambda z.\, zx) \rightarrow$
$(\lambda z.\, zx)((\lambda z.\, zx)y) \rightarrow$
$(\lambda z.\, zx)(yx) \rightarrow$
$(yx)x$

Codifiche in λ-calcolo

Il λ-calcolo, per come visto finora, è molto astratto e molto limitato. Nonostante ciò, è molto potente ed espressivo:

- è Turing completo
- i booleani, le coppie, le costanti e le operazioni aritmetiche possono essere espresse tramite il λ-calcolo, vediamo come:

- Booleani

- True = $\lambda x.\, \lambda y.\, x$
- False = $\lambda x.\, \lambda y.\, y$

- Costrutto `if`

- If a then b else c = a b c

> **Esempio**
>
> - If true then b else c = $(\lambda x.\, \lambda y.\, x)b\ c \rightarrow (\lambda y.\, b)c \rightarrow b$
> - If false then b else c = $(\lambda x.\, \lambda y.\, y)b\ c \rightarrow (\lambda y.\, y)c \rightarrow c$

Operatori booleani

- Not: $\lambda x.\ x\ false\ true$
- And: $\lambda x.\ \lambda y.\ xy\ false$
- Or: $\lambda x.\ \lambda y.\ x\ true\ y$

Coppie

Codifica di una coppia (a, b):

- $(a, b) = \lambda x.\ if\ x\ then\ a\ else\ b$
- $first = \lambda f.\ f\ true$
- $second = \lambda f.\ f\ false$

Numeri naturali

Per codificare i numeri naturali si fa uso degli assiomi di Peano:

- 0 è un numero naturale
- se `n` è un numero naturale, anche `succ(n)` lo è
  Dunque si codifica 0 come $\lambda f.\ \lambda x.\ x$, e poi si applica la funzione `succ()` n volte per arrivare ad ottenere n. In particolare, n è rappresentato tramite la funzione di ordine superiore che mappa una funzione f alla sua composizione n volte.
  Ricapitolando, si ha che:
- $0 = \lambda f.\ \lambda x.\ x$
- $1 = \lambda f.\ \lambda x.\ f\ x$
- $2 = \lambda f.\ \lambda x.\ f(f\ x)$
- $3 = \lambda f.\ \lambda x.\ f(f(f\ x))$
- $4 = \lambda f.\ \lambda x.\ f(f(f(f\ x)))$
- ...
  La funzione `succ()` viene così definita: $succ(n) = \lambda n.\ \lambda f.\ \lambda x.\ f(n\ f\ x)$, in quanto prende $n\ f\ x$, che è f applicata n volte a x, e ci applica f ancora una volta.

Operazioni sui numeri naturali

- verifica che un numero sia zero: $\lambda z.\ z(\lambda y.\ false)true$
- addizione: se n significa "*f viene applicata n volte a x*", allora n+m significa "*applica f n volte al risultato dell'applicazione di f m volte a x*". Da cui si ricava: $\lambda n.\ \lambda m.\ \lambda f.\ \lambda x.\ nf(mfx)$

Considerazioni finali

La sintassi del λ-calcolo è complessa. Perciò è possibile fare un leggero abuso di notazione, utilizzando numero, operazioni ed espressioni, come nel caso di $\lambda x.\ (x + 2)$ o $\lambda x.\ if\ x = 1\ then$, dove vengono usate come abbreviazioni di espressioni del λ-calcolo puro.

Ricorsione

Supponiamo di voler scrivere la funzione ricorsiva per il calcolo del fattoriale.
$\lambda n.\, if\ (n = 0)\ then\ 1\ else\ (n * (f(n-1)))$
Questa scrittura non funziona, perché non possiamo definire quale sia la variabile f, dal momento che non possiamo dare un nome alle funzioni.
Scriviamo allora la funzione come:
$G = \lambda f.\, \lambda n.\, if\ n = 0\ then\ 1\ else\ n * f(n-1)$
Vogliamo allora trovare una funzione f tale che $f = G(f)$, dove G è una funzione di ordine superiore che prende una funzione come argomento e ritorna una funzione.
Un modo per calcolare tale funzione è la Y combinator: $\lambda f.\, (\lambda x.\, f(xx))(\lambda x.\, f(xx))$
La Y combinator è una funzione che applicata a G ritorna la funzione f che stavamo cercando.

19. Laboratorio 9

Esercizio 9.1

Dato il tipo `tree`, per la rappresentazione di un generico albero, scrivere una funzione `isOn` che, dato un elemento `x` e un albero generico `T`, dice se `x` appare tra le label di `T`.

```
> datatype ('label) tree =
    Node of 'label * 'label tree list;

> fun isOn x (Node(y, nil)) = if x = y then true else false
    | isOn x (Node(y, u::ux)) = (isOn x (Node(y, ux)) orelse (isOn x u));

> isOn 3 (Node(2, [Node (3,nil), Node(5,nil)]));

(* datatype 'a tree = Node of 'a * 'a tree list
val isOn = fn: ''a -> ''a tree -> bool
val it = true: bool
val it = (): unit *)
```

Esercizio 9.2

Dato il tipo `tree`, per la rappresentazione di un generico albero, scrivere una funzione `count(T, x)` che, dato un albero `T` e un elemento `x`, ritorna il numero di occorrenze di `x` tra le label di `T`.

```
> datatype ('label) tree =
    Node of 'label * 'label tree list;

> fun count (Node(y, nil)) x = (if x = y then 1 else 0)
    | count (Node(y, u::ux)) x = ((count u x) + (count (Node(y, ux)) x));

> count(Node(2, [Node(3,nil), Node(2,nil)])) 2;

(* datatype 'a tree = Node of 'a * 'a tree list
val count = fn: ''a tree -> ''a -> int
val it = 2: int
val it = (): unit *)
```

Esercizio 9.3

Dato il tipo `tree`, per la rappresentazione di un generico albero, scrivere una funzione `depth` che, dato un albero `T`, ritorna la profondità di `T`.

```
> datatype ('label) tree =
    Node of 'label * 'label tree list;

> fun depth (Node(y, nil)) = 1
    | depth (Node(y, u::ux)) =
      let
        val t1 = depth (Node(y, ux))
        val t2 = depth u + 1
      in
        if t1 > t2 then t1 else t2
      end;

> depth (Node(2, [Node(3, [Node(4, nil)]), Node(2, nil)]));

(* datatype 'a tree = Node of 'a * 'a tree list
val depth = fn: 'a tree -> int
val it = 3: int
val it = (): unit *)
```

Esercizio 9.4

Dato il tipo `tree`, per la rappresentazione di un generico albero, scrivere una funzione `preOrder` che, dato un albero `T`, ritorna la lista dei nodi di `T` in pre-ordine.

```
> datatype ('label) tree =
    Node of 'label * 'label tree list;

> fun preOrder (Node(0, nil)) = []
    | preOrder (Node(y, nil)) = [y]
    | preOrder (Node(0, u::ux)) = (preOrder u) @ (preOrder (Node(0, ux)))
    | preOrder (Node(y, u::ux)) = [y] @ (preOrder u) @ (preOrder (Node(0,
ux)));

> preOrder(Node(2, [Node(3, [Node(4, nil)]), Node(2, nil)]));

(* datatype 'a tree = Node of 'a * 'a tree list
val preOrder = fn: int tree -> int list
```

```
val it = [2, 3, 4, 2]: int list
val it = (): unit *)
```

Esercizio 9.5

Definire una structure `Tree` contenente il datatype `tree` per la rappresentazione di un albero generico. La structure deve contenere:

- `create(a)`, che ritorna un nodo dell'albero con label a
- `build(a, L)`, che ritorna un albero con la radice che ha label a e ha L come lista dei sottoalberi
- `subtree(i, T)`, che trova l'i-esimo sottoalbero della radice, con tanto di eccezione `Missing` se questo non esiste.
  Costruire un albero `myTree` usando le funzioni della structure e testarne le funzionalità.

```
> structure Tree = struct
    datatype ('label) tree =
      Node of 'label * 'label tree list

    exception Missing

    fun create(a) = Node(a, nil)

    fun build(a, L) = Node(a, L)

    fun subtree(i, (Node(a, nil))) = raise Missing
      | subtree(1, (Node(a, L))) = L
      | subtree(i, (Node(a, x::xs))) = subtree(i-1, (Node(a,xs)))
end;

> val v4 = Tree.create(4);
> val v3 = Tree.build(3, [v4]);
> val v2 = Tree.create(2);
> val v1 = Tree.build(2, [v3, v2]);

> Tree.subtree(2, v1);

(*structure Tree:
  sig
    exception Missing
    val build: 'a * 'a tree list -> 'a tree
    val create: 'a -> 'a tree
    val subtree: int * 'a tree -> 'a tree list
    datatype 'a tree = Node of 'a * 'a tree list
  end
```

```
val v4 = Node (4, []): int Tree.tree
val v3 = Node (3, [Node (4, [])]): int Tree.tree
val v2 = Node (2, []): int Tree.tree
val v1 = Node (2, [Node (3, [Node (4, [])]), Node (2, [])]): int Tree.tree
val it = [Node (2, [])]: int Tree.tree list
val it = (): unit *)
```

Esercizio 9.6

Nell'esercizio precedente, `create(a)` corrisponde a fare `build(a, nil)`. Definire una nuova structure `SimpleTree` che ha le stesse funzionalità di `Tree` tranne `create()` e che pone le label tutte di interi.

```
> signature SIMPLE = sig
    datatype 'label tree =
      Node of 'label * 'label tree list

    exception Missing

    val build: int * int tree list -> int tree
    val subtree: int * int tree -> int tree
  end;

> structure Tree = struct
    datatype ('label) tree =
      Node of 'label * 'label tree list

    exception Missing

    fun create(a) = Node(a, nil)

    fun build(a, L) = Node(a, L)

    fun subtree(i, (Node(a, nil))) = raise Missing
      | subtree(1, (Node(a, x::xs))) = x
      | subtree(i, (Node(a, x::xs))) = subtree(i-1, (Node(a,xs)))
  end;

> structure SimpleTree : SIMPLE = Tree;

(* signature SIMPLE =
  sig
    exception Missing
    val build: int * int tree list -> int tree
    val subtree: int * int tree -> int tree
    datatype 'a tree = Node of 'a * 'a tree list
  end
```

```
structure Tree:
  sig
    exception Missing
    val build: 'a * 'a tree list -> 'a tree
    val create: 'a -> 'a tree
    val subtree: int * 'a tree -> 'a tree
    datatype 'a tree = Node of 'a * 'a tree list
  end
structure SimpleTree: SIMPLE
val it = (): unit *)
```

Esercizio 9.7

Utilizzare la funzione `build()` di `SimpleTree` per costruire un albero con radice con label 1 e tre figli con label 2,3,4. Applicare `subtree` per ottenere il secondo subtree della radice.

```
> val root = SimpleTree.build(1, [SimpleTree.build(2, []),
SimpleTree.build(3, []), SimpleTree.build(4, [])]);

> val s2 = SimpleTree.subtree(2, root);

(* val root = Node (1, [Node (2, []), Node (3, []), Node (4, [])]):
   int SimpleTree.tree
val s2 = Node (3, []): int SimpleTree.tree
val it = (): unit *)
```

Esercizio 9.8

Definire una structure `Stack` che rappresenta uno stack di elementi di un tipo arbitrario. Includere le funzioni:

- `create`, per creare uno stack vuoto
- `push x s`, che inserisce `x` in cima allo stack
- `pop s`, che ritorna lo stack senza il primo elemento oppure alza un'eccezione `EmptyStack`
- `isEmpty s`, che controlla se lo stack è vuoto
- `top s`, che ritorna l'elemento in cima allo stack o alza `EmptyStack`.

```
> structure Stack = struct
    type 'a stack = 'a list;
    exception EmptyStack;
    val emptyStack = [];
```

```
    fun create() = emptyStack;

    fun push x s = (x::s);

    fun pop nil = raise EmptyStack
      | pop (x::xs) = xs;

    fun isEmpty nil = true
      | isEmpty s = false;

    fun top nil = raise EmptyStack
      | top (x::xs) = x;
  end;

(* structure Stack:
  sig
    exception EmptyStack
    val create: unit -> 'a list
    val emptyStack: 'a list
    val isEmpty: 'a list -> bool
    val pop: 'a list -> 'a list
    val push: 'a -> 'a list -> 'a list
    eqtype 'a stack
    val top: 'a list -> 'a
  end
val it = (): unit *)
```

Esercizio 9.9

Si realizzi una signature che permetta di creare una structure `StringStack`, i cui elementi sono ti dipo string e che omette l'operazione `top`.

```
> signature StringStack = sig
    type 'a stack
    exception EmptyStack
    val emptyStack: string list
    val create: unit -> string stack
    val push: string -> string stack -> string stack
    val pop: string stack -> string stack
    val isEmpty: string stack -> bool
  end;

(* signature StringStack =
  sig
    exception EmptyStack
    val create: unit -> string stack
```

```
    val emptyStack: string list
    val isEmpty: string stack -> bool
    val pop: string stack -> string stack
    val push: string -> string stack -> string stack
    type 'a stack
  end
val it = (): unit *)
```

Esercizio 9.10

Realizzare una structure `Queue` che rappresenta una cosa di elementi. Includere:

- `create`, per creare una coda vuota
- `EmptyQueue`, un'eccezione
- `Enqueue(x, Q)`, che aggiunge un elemento alla fine e ritorna la coda
- `Dequeue Q`, che ritorna la coppa del primo elemento della cosa e il resto della cosa. Se è vuota, alza `EmptyQueue`
- `isEmpty Q`, che controlla se la coda è vuota

```
> structure Queue = struct
    type 'a queue = 'a list;

    exception EmptyQueue;

    val create = nil;

    fun Enqueue(x, Q) = Q @ [x];

    fun Dequeue nil = raise EmptyQueue
      | Dequeue Q = (hd(Q), tl(Q));

    fun isEmpty nil = true
      | isEmpty Q = false;
  end;

(* structure Queue:
  sig
    val Dequeue: 'a list -> 'a * 'a list
    exception EmptyQueue
    val Enqueue: 'a * 'a list -> 'a list
    val create: 'a list
    val isEmpty: 'a list -> bool
    eqtype 'a queue
  end
val it = (): unit *)
```

Esercizio 9.11

Realizzare una signature per creare una structure `PairQueue` di coppie formate da una string e un intero, con le stesse operazioni di `Queue`.

```
> signature PAIR_QUEUE = sig
    type 'a queue;
    exception EmptyQueue;
    val create: (string * int) queue;
    val Enqueue: ((string * int) * (string * int) queue) -> (string * int)
queue;
    val Dequeue: (string * int) queue -> ((string * int) * (string * int)
queue);
    val isEmpty: (string * int) queue -> bool
  end;

(* signature PAIR_QUEUE =
  sig
    val Dequeue:
       (string * int) queue -> (string * int) * (string * int) queue
    exception EmptyQueue
    val Enqueue:
       (string * int) * (string * int) queue -> (string * int) queue
    val create: (string * int) queue
    val isEmpty: (string * int) queue -> bool
    type 'a queue
  end
val it = (): unit *)
```

Esercizio 9.12

Scrivere una structure `IntTriple` che implementa la signature `TOTALORDER` che, al posto di `string`, definisce gli elementi di un albero binario di ricerca come tuple di 3 interi.
lt: (a, b, c) < x, y, z) if:

- a < x
- a = x and b < y
- a = x, b = y and c < z
  Usare il funtore `MakeBST` per ottenere una structure che conserva le triple di interi in alberi binari.

```
> signature TOTALORDER = sig
    type element;
```

```
    val lt : element * element -> bool
  end;
(* signature TOTALORDER = sig type element val lt: element * element ->
bool end; *)
*
functor MakeBST(Lt: TOTALORDER):
    sig
        type 'label btree;
        exception EmptyTree;
        val create: Lt.element btree;
        val lookup: Lt.element * Lt.element btree -> bool;
        val insert: Lt.element * Lt.element btree -> Lt.element btree;
        val deletemin : Lt.element btree -> Lt.element * Lt.element
btree;
        val delete : Lt.element * Lt.element btree -> Lt.element btree
    end
    =
    struct
        open Lt;
        datatype 'label btree
            = Empty
            | Node of 'label * 'label btree * 'label btree;
        val create = Empty;
        fun lookup(x, Empty) = false
            | lookup(x, Node(y, left, right)) =
                if lt(x, y)
                then lookup(x, left)
                else if lt(y, x)
                    then lookup(x, right)
                    else (* x=y *) true;
        fun insert(x, Empty) = Node(x, Empty, Empty)
          | insert(x, T as Node(y, left, right)) =
            if lt(x, y)
            then Node(y, insert(x, left), right)
            else
                if lt(y, x)
                then Node(y, left, insert(x, right))
                else (* x=y *) T; (* do nothing; x was already there *)
        exception EmptyTree;
        fun deletemin(Empty) = raise EmptyTree
          | deletemin(Node(y, Empty, right)) = (y, right)
          | deletemin(Node(w, left, right)) = let
                val (y, L) = deletemin(left)
            in
                (y, Node(w, L, right))
            end;
        fun delete(x, Empty) = Empty
          | delete(x, Node(y, left, right)) =
            if lt(x, y)
            then Node(y, delete(x, left), right)
```

```
              else if lt(y, x)
                   then Node(y, left, delete(x, right))
                   else (* x=y *) case (left, right)
                        of (Empty, r) => r
                        | (l, Empty) => l
                        | (l, r) => let
                        val (z, r1) = deletemin(r)
                        in
                             Node(z, l, r1)
                        end;
end;

structure IntTriple: TOTALORDER = struct
     type element = int * int * int;
     fun lt((l1, l2, l3), (r1, r2, r3)) =
     l1<r1 orelse l1=r1 andalso l2<r2 orelse l1=r1 andalso l2=r2 andalso
l3<r3;
end;

(* structure IntTriple: TOTALORDER *)

structure IntTripleBST = MakeBST (IntTriple);
```